

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий
Дисциплина: Компьютерные системы и сети (КСИС)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту на тему

«Battleship Online»

БГУИР КП 6-05-0612-01 023 ПЗ

Выполнил
студент: гр. 351004

Наривончик А.М.

Проверил:

Базылев Е.Н.

Минск 2025

СОДЕРЖАНИЕ

Введение.....	5
1 Анализ предметной области	6
1.1 Обзор существующих аналогов.....	6
1.2 Перечень функциональных требований	8
2 Моделирование предметной области.....	9
2.1 Архитектура «клиент-сервер»	9
2.2 Технология SignalR для модели клиента и сервера.....	9
3 Проектирование программного средства	11
3.1 Структура серверной части программы	11
3.2 Проектирование интерфейса программного средства	11
3.3 Проектирование функционала программного средства	15
4 Разработка программного средства.....	22
4.1 Серверная часть: API для онлайн игры	22
4.2 Клиентская часть: конструктор поля	24
4.3 Клиентская часть: игровой процесс	26
5 Тестирование программного средства.....	30
5.1 Запуск сервера и подключение клиента	30
5.2 Игровой процесс.....	31
5.3 Чат.....	31
6 Руководство пользователя.....	32
6.1 Начало работы в «Battleship Online».....	32
6.2 Страница расстановки кораблей.....	32
6.3 Страница игрового процесса.....	33
6.4 Правила игры	34
Заключение	36
Список использованной литературы.....	37
Приложение А	38

ВВЕДЕНИЕ

Сколько существует человечество – столько существуют игры. Еще древние люди на досуге занимали себя при помощи обрывков ткани, красивых камней, коряг. Креативности человека никогда не было предела, благодаря чему были изобретены миллионы способов провести время с удовольствием.

Бывают игры, способные одним только упоминанием окунуть любого взрослого в детство. Одна из таких игр носит название «Морской бой».

«Морской бой» – это стратегия, требующая от игроков внимательности, умения анализировать обстановку на поле и тактического мышления. Целью игры является полное уничтожение вражеского флота. Сам флот представляет собой 10 кораблей (1 четырехпалубный, 2 трёхпалубных, 3 двухпалубных и 4 однопалубных), расположенных на игровом поле, размером 10x10. За один ход игрок может сделать один выстрел по игровому полю соперника. В случае попадания корабль соперника будет ранен или потоплен полностью.

«Морской бой» был придуман в 1870-х годах Петром Кондратьевым, вынужденным работать бурлаком на реке Каме. Однажды он предложил товарищам по несчастью игру. Идея бурлакам пришлась настолько по душе, что спустя столько лет люди всё так же азартно топят корабли противников!

В тридцатые годы прошлого века в США фирмой Starex начался выпуск разграфлённых блокнотов для игры, а уже в шестидесятых годах всё там же, в Америке, Милтон Брэдли основал компанию The Milton Bradley Company. Именно тогда с конвейера сошла версия игры, которая использовала пластмассовые доски, а к ним маленькие боевые кораблики и фишки, обозначающие промахи или попадания. Игра позиционировала себя как солидный центр управления флотом. Это уже не обычное чирканье ручки по листочку в клетку. Это масштабные военные действия, и каждый игрок – не меньше, чем сам адмирал! Через какое-то время на рынке появилась электронная версия «Морского боя», где при попадании в корабль противника раздавался звук взрыва и это действие ознаменовывалось вспыхивающим светом! Сегодня, в эпоху доступности электронных гаджетов, игра получила большое количество реализация для всевозможных платформ.

Целью данной курсовой работы является разработка программного средства «Battleship Online» с использованием веб-технологий.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих аналогов

Игра «Морской бой» очень известна. За долгие годы существования она успела приобрести много разных форм: существует бумажная версия игры, «Морской бой» на пластмассовом поле с фишками, игра разработана для компьютеров и смартфонов. «Морской бой» богат режимами игры, добавляющими разнообразия игровому процессу.

1.1.1 «Морской бой онлайн»

Это вариант игры с сайта <https://battleship-game.org/ru/>. Игра ведется по классическим правилам. Сначала программа подберет пользователю соперника, затем предложит расставить свои корабли вручную или сделать это автоматически, после чего реализуется сам процесс игры. Из удобного в этой версии можно отметить подсветку выбранной пользователем ячейки для выстрела и не перегруженный, компактный интерфейс. Внешний вид программы представлен на рисунке 1.1.

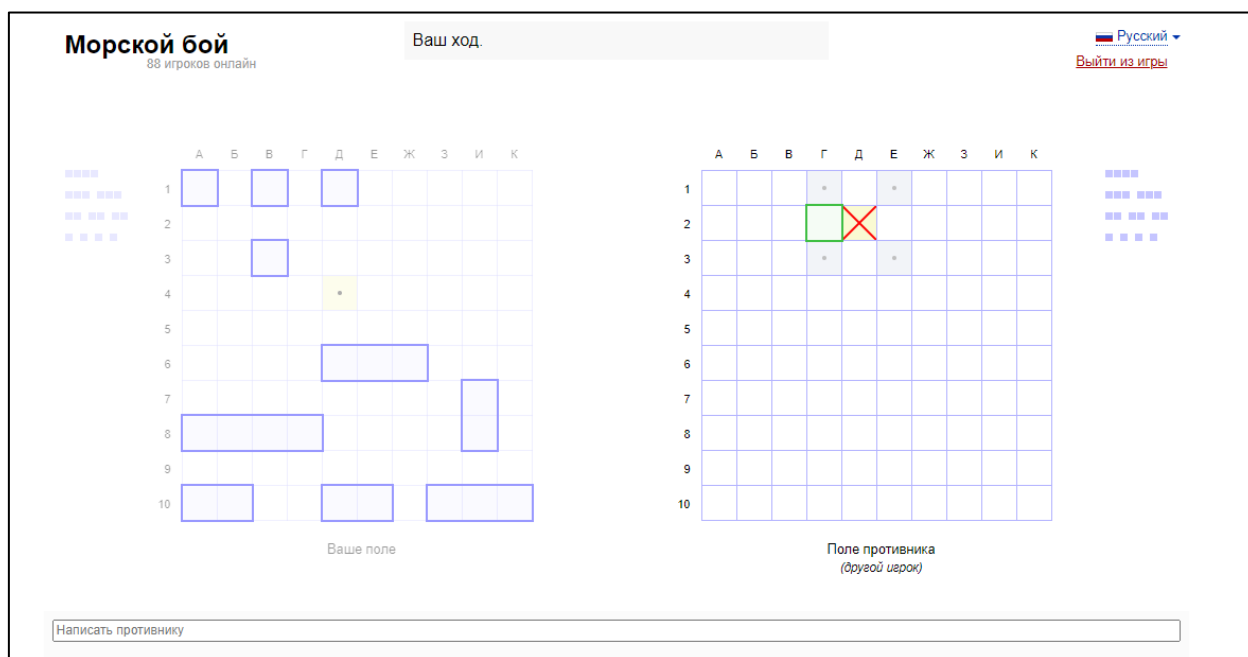


Рисунок 1.1 – внешний вид игры «Морской бой онлайн»

1.1.2 «Морской бой классический»

Данную версию игры можно найти на сайте <https://yandex.by/games/app/214025>. Игра также ведется по классическим правилам. Стоит отметить удобный конструктор поля кораблей пользователя с возможностью расстановки кораблей автоматически, с подсветкой ячеек на месте устанавливаемого корабля. Процесс игры существенно упрощают подсказки. Внешний вид программы представлен на рисунке 1.2.

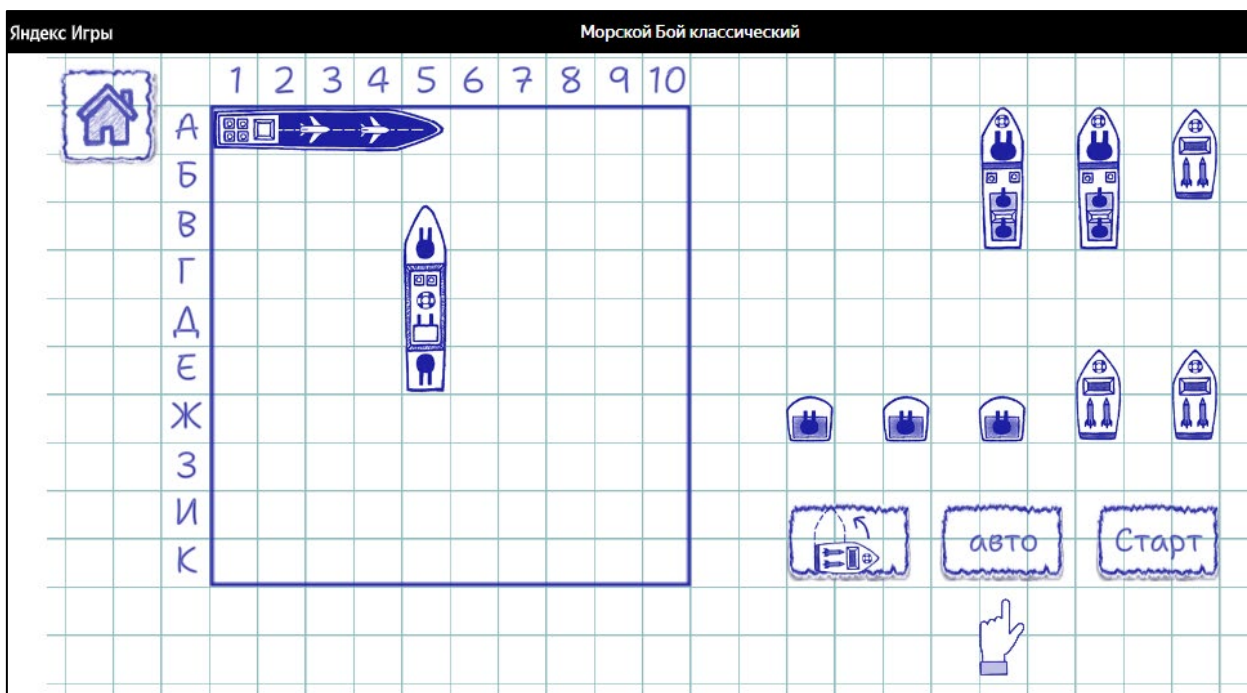


Рисунок 1.2 – внешний вид игры «Морской бой классический»

1.1.3 «Sea Battle 2»

Этот вариант «Морского боя» выпущен для Android и содержит в себе 2 режима игры – классический и расширенный. В расширенном режиме игра предоставляет возможность вести сражение не только с помощью кораблей, но и самолётов. Игра содержит в себе сюжетную линию с обучением, а также возможность сыграть как с компьютером, так и с реальным игроком онлайн или на одном устройстве. Внешний вид программы представлен на рисунке 1.3.

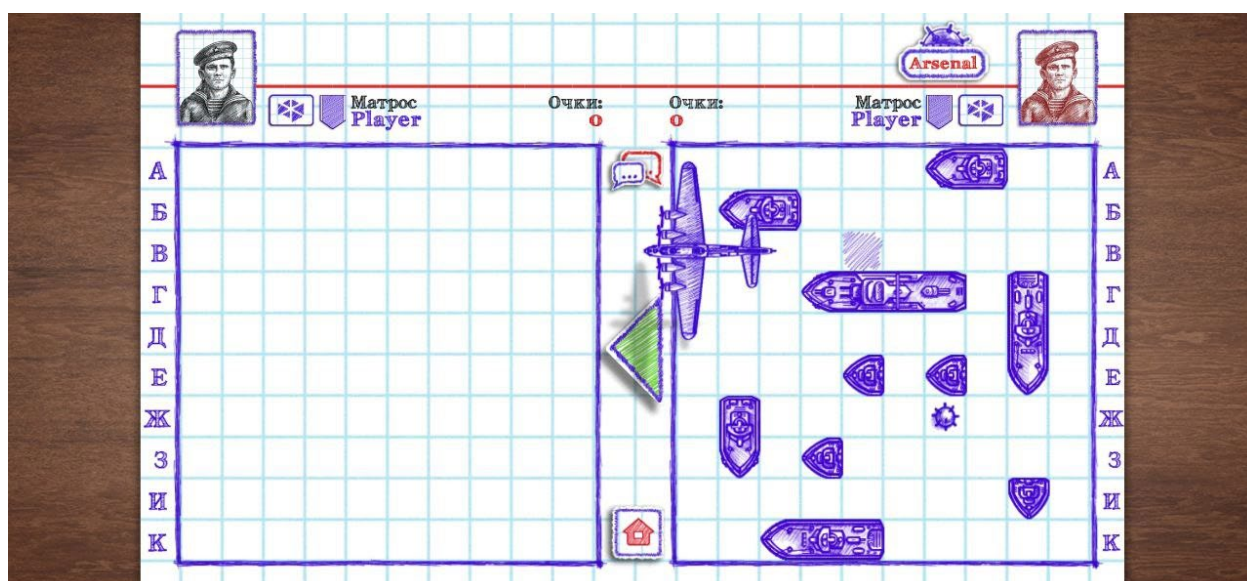


Рисунок 1.3 – внешний вид игры «Sea Battle 2»

1.2 Перечень функциональных требований

После анализа аналогов был составлен следующий список требуемых функций:

Игровой процесс:

- игра должна поддерживать классические правила "Морского боя" (размещение кораблей, поочередные ходы, определение победителя).
- игроки должны иметь возможность размещать корабли на своем поле перед началом игры.
- система должна автоматически проверять корректность расстановки кораблей.
- после начала игры игроки должны поочередно совершать ходы, а система — обрабатывать попадания и промахи.
- игра должна завершаться при уничтожении всех кораблей одного из игроков с определением победителя.

Мультиплеер:

- игра должна поддерживать онлайн-режим для двух игроков через интернет.
- должна быть реализована система поиска соперника (лобби или автоматический подбор).
- игроки должны видеть статус игры (ожидание хода, ваш ход, завершение игры).

Пользовательский интерфейс:

- должны быть реализованы экраны:
- главное меню (вход, регистрация, выбор режима игры).
- лобби для поиска соперника.
- игровое поле с визуализацией своих кораблей и ходов противника.

Технологический стек:

- frontend: React с TypeScript и системой сборки Vite.
- backend: ASP.NET Core для реализации игровой логики и API.
- сетевое взаимодействие: SignalR для обмена данными между клиентом и сервером в реальном времени.

Данные требования обеспечивают создание современной онлайн-версии "Морского боя" с удобным интерфейсом, стабильным сетевым взаимодействием.

2 МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1 Архитектура «клиент-сервер»

Архитектура «клиент-сервер» представляет собой модель взаимодействия в распределённых системах, где роли участников разделены на поставщиков ресурсов (серверы) и потребителей (клиенты). Серверы обрабатывают запросы от клиентов, предоставляя им данные или функциональность через сетевые протоколы. Клиенты, такие как веб-браузеры или мобильные приложения, инициируют запросы и используют полученные ресурсы. Например, при загрузке веб-страницы браузер отправляет запрос серверу, который возвращает необходимые данные.

Преимущества архитектуры:

- возможность одновременного обслуживания множества клиентов;
- централизованное хранение данных, упрощающее управление и безопасность;
- организация взаимодействия между клиентами через сервер.

Недостатки:

- зависимость клиентов от доступности сервера;
- риск перегрузки сервера при высокой нагрузке;
- уязвимость к кибератакам, направленным на серверную инфраструктуру.

2.2 Технология SignalR для модели клиента и сервера

SignalR — это библиотека для ASP.NET Core, предназначенная для реализации веб-сокетов и других технологий двусторонней связи в реальном времени между клиентом и сервером. В отличие от HTTP, который работает по схеме "запрос-ответ", SignalR обеспечивает постоянное соединение, позволяя серверу отправлять данные клиенту без необходимости явного запроса.

Поддержка нескольких транспортных протоколов:

- WebSocket (наиболее эффективный, бинарный протокол с минимальными задержками).
- Server-Sent Events (SSE) (односторонняя отправка данных от сервера).
- Long Polling (эмуляция реального времени через периодические HTTP-запросы).

SignalR автоматически выбирает оптимальный транспорт в зависимости от возможностей клиента и сервера.

Режим работы:

- хаб (Hub) — основной механизм взаимодействия, позволяющий вызывать методы на сервере из клиента и наоборот.

- RPC-подход (Remote Procedure Call) — сервер и клиент могут вызывать друг у друга методы, как в локальном приложении.

Преимущества перед HTTP для онлайн-игр:

- минимальные задержки (особенно при использовании WebSocket).
- эффективная передача данных (меньше накладных расходов по сравнению с HTTP).
- автоматическое управление соединением (переподключение при обрыве, поддержка разных транспортов).

В проекте Battleship Online SignalR применяется для:

- синхронизации игровых событий (ходы игроков, попадания, завершение игры).
- организации мультиплеера (поиск соперника, создание лобби, обновление состояния игры).
- отправки уведомлений (например, о подключении противника или его ходе).

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

3.1 Структура серверной части программы

Серверная часть Battleship Online построена на основе ASP.NET Core с использованием технологии SignalR для обеспечения реального времени взаимодействия между игроками. Основу серверной логики составляет класс SeaBattleHub, который наследуется от Hub и обрабатывает все игровые события.

В основе работы сервера лежит система управления игровыми сессиями. При подключении игрока он может присоединиться к очереди ожидания через метод JoinQueue. Как только в очереди накапливается два игрока, система автоматически создает новую игровую сессию (GameSession) и уведомляет обоих участников о начале матча.

Для хранения активных игровых сессий используется потокобезопасная коллекция ConcurrentDictionary, где ключом выступает уникальный идентификатор сессии, а значением - объект GameSession, содержащий всю необходимую информацию о текущем состоянии игры. Этот объект включает идентификаторы подключений обоих игроков, их игровые поля, информацию о текущем ходе, а также дополнительные данные, такие как имена участников.

Особое внимание уделено обработке расстановки кораблей. Когда игрок отправляет свое игровое поле через метод SendBattlefield, сервер сохраняет эту информацию и передает ее сопернику. После получения полей от обоих игроков система автоматически начинает игру, уведомляя участников о старте матча.

Механика ходов реализована через метод MakeMove, который принимает координаты выстрела и передает их противнику. Сервер также поддерживает чат между игроками через метод SendChatMessage, позволяя участникам обмениваться сообщениями во время игры.

Важной особенностью реализации является надежная обработка отключений игроков. При разрыве соединения система автоматически удаляет игрока из очереди ожидания (если он там находился) или завершает текущую игровую сессию, уведомляя второго участника о преждевременном окончании матча.

3.2 Проектирование интерфейса программного средства

3.2.1 Главное меню

Главное меню приложения Battleship Online представляет собой лаконичное и интуитивно понятное меню, выполненное в морской стилистике с использованием сине-голубой цветовой палитры. На фоне изображения расположены две основные кнопки-якоря, предлагающие выбор режима игры:

"Игра по сети" - кнопка для запуска многопользовательского режима через интернет;

"Игра с ботом" - кнопка для начала одиночной игры против компьютерного противника.

Макет страницы представлен на рисунке 3.1.

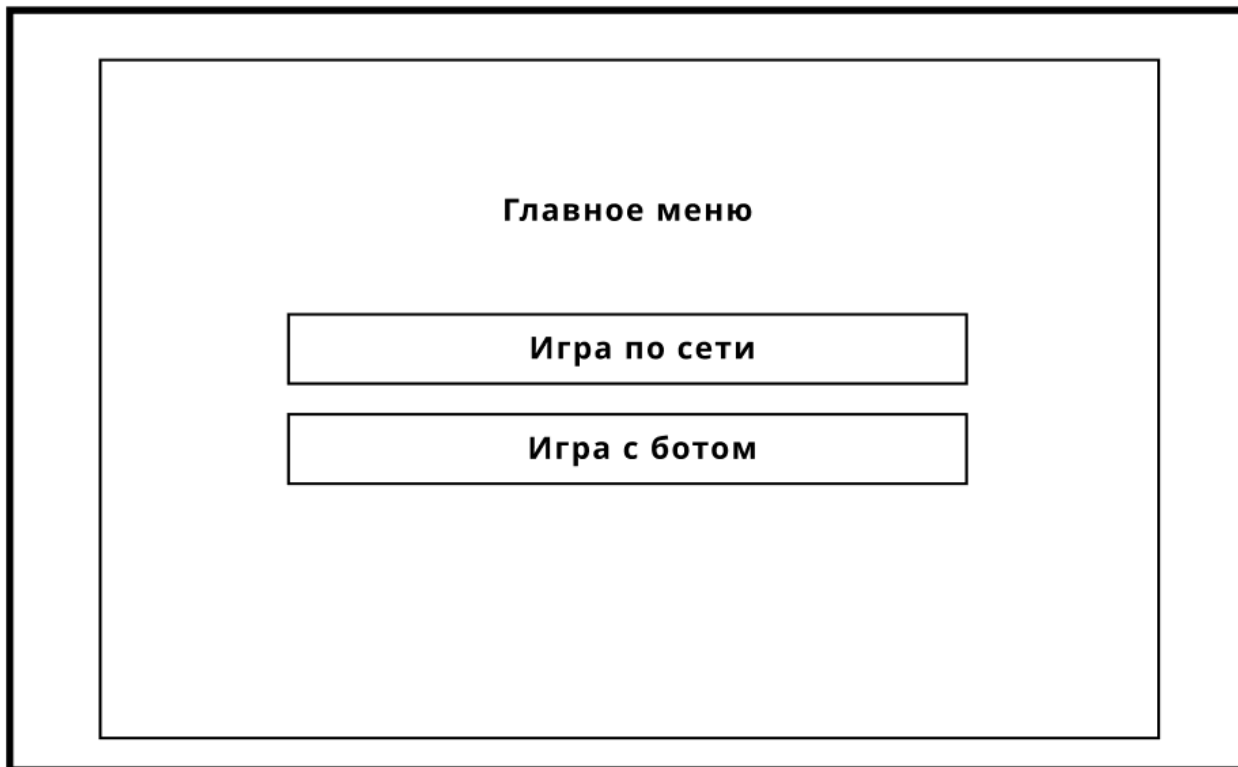


Рисунок 3.1 – Главное меню

3.2.2 Страница расстановки кораблей

Экран подготовки к игре в Battleship Online содержит несколько ключевых элементов. В верхней части расположено поле для ввода игрового псевдонима. Ниже отображаются доступные для расстановки корабли различных размеров.

Интерфейс включает функциональную кнопку "Авторасстановка", которая автоматически размещает корабли на игровом поле стандартного размера 10×10 клеток. Основная область экрана отведена под интерактивное игровое поле с координатной сеткой.

После корректного размещения всех кораблей активируется кнопка "Начало игры". Визуальное оформление выдержано в морской тематике с использованием синей цветовой гаммы и стилизованных изображений кораблей. Интерфейс обеспечивает интуитивно понятное взаимодействие и адаптируется под различные размеры экранов. Макет представлен на рисунке 3.2.



Рисунок 3.2 – Страница расстановки кораблей

3.2.3 Страница игры с ботом

Игровой экран Battleship Online представляет собой интерактивное пространство, разделенное на две основные зоны. Слева располагается поле игрока, где отображаются собственные корабли и результаты атак противника - попадания отмечаются красными квадратами, промахи белыми кругами. Справа находится поле соперника, на котором игрок отмечает свои атаки, при этом реальное расположение вражеских кораблей остается скрытым до их полного уничтожения.

В нижней части экрана выводится текущий статус игры, включая информацию о ходе, оставшееся время на ход (если включен таймер) и счет уничтоженных кораблей. Статусная строка обновляется в реальном времени, отражая все изменения игровой ситуации. В углу экрана размещена кнопка выхода в главное меню.

Визуальное оформление сохраняет морскую тематику - синие оттенки фона имитируют водную поверхность. Макет страницы представлен на рисунке 3.3.

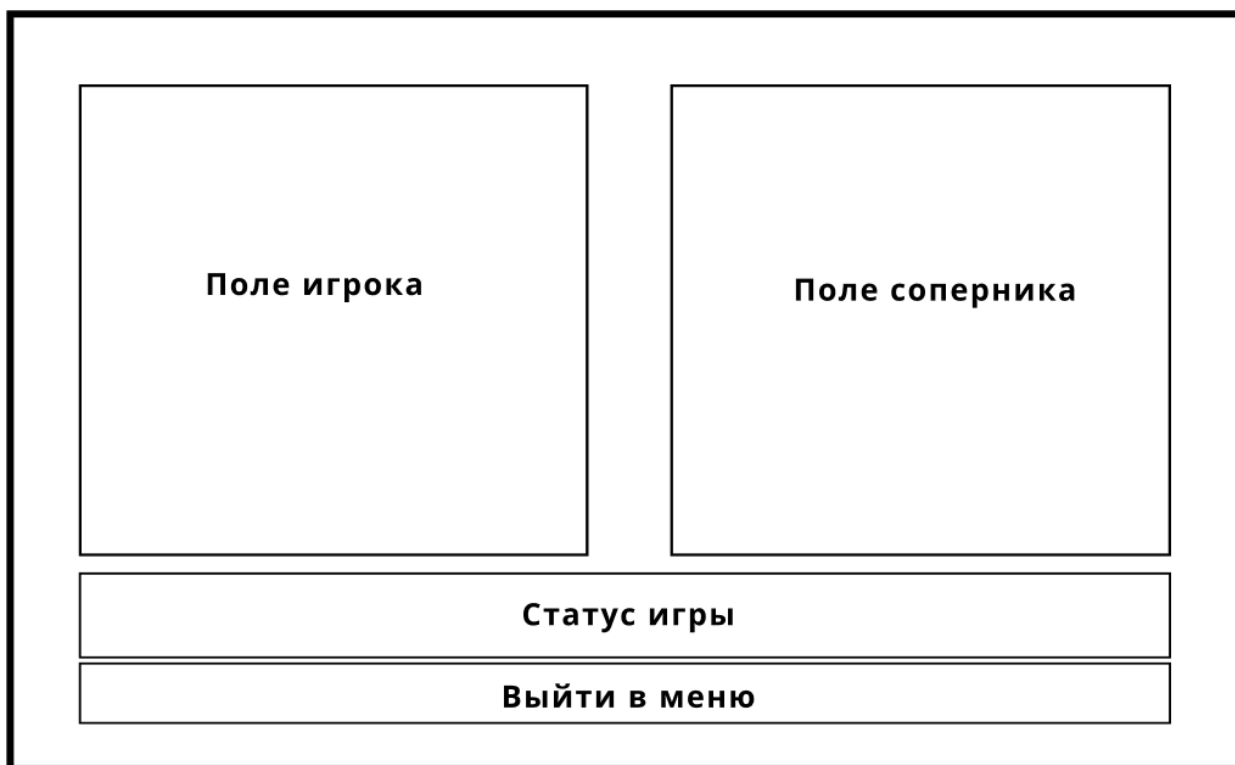


Рисунок 3.3 – Страница игры с ботом

3.2.4 Страница онлайн-игры

Экран игрового процесса онлайн состоит из нескольких ключевых элементов. Основное пространство разделено между игровыми полями: слева расположено собственное поле игрока с отображением кораблей и попаданий противника, справа - поле соперника для нанесения атак. Справа от полей находится чат для обмена сообщениями с оппонентом.

В нижних части интерфейса отображается текущий статус игры, включая информацию о ходе игры. Статусная панель динамически обновляется, отражая изменения в ходе сражения.

В нижнем углу экрана расположена кнопка выхода в главное меню. Интерфейс выполнен в морской стилистике с преобладанием синих тонов. Игровые поля представлены в виде координатных сеток 10×10 с традиционной разметкой. Визуальные элементы включают анимированные эффекты волн и всплесков при попаданиях, что добавляет динамики игровому процессу, сохраняя при этом четкость и удобство восприятия информации.

Макет представлен на рисунке 3.4.

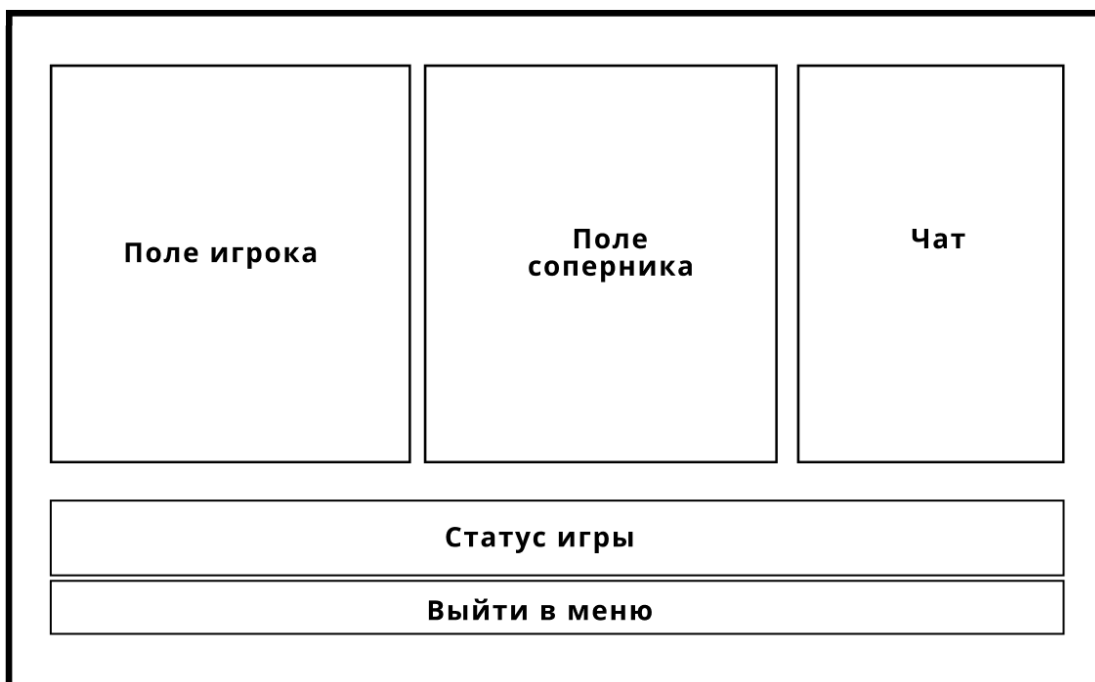


Рисунок 3.4 – Страница онлайн-игры

3.3 Проектирование функционала программного средства

При разработке Battleship Online необходимо реализовать следующие ключевые алгоритмы:

- конструктор игрового поля;
- генерация случайных расстановок кораблей;
- обработка выстрела;
- запуск и управление сетевой игрой.

Каждый из этих алгоритмов требует четкой проработки внутренней логики и безупречной синхронизации между клиентской и серверной частью. Особое внимание следует уделить механизмам обработки ходов и сетевого взаимодействия, которые являются критически важными для обеспечения плавного игрового процесса. Грамотно реализованные алгоритмы станут основой стабильной работы всех систем игры.

3.3.1 Конструктор игрового поля

Процесс расстановки кораблей в Battleship Online реализован через последовательный алгоритм взаимодействия игрока с системой. Изначально пользователь выбирает конкретный корабль из числа доступных для размещения. После выбора система анализирует текущее состояние игрового поля и определяет возможные варианты расположения выбранного корабля с учетом установленных правил - соблюдения границ поля, отсутствия пересечений с уже размещенными кораблями и обязательных промежутков между ними.

На основе этих расчетов интерфейс визуально выделяет допустимые позиции для размещения носа корабля. Игрок указывает конкретную ячейку и направление (горизонтальное или вертикальное), после чего система окончательно фиксирует корабль на поле. Весь процесс повторяется до полной расстановки всего флота согласно игровым правилам. В любой момент игрок может сбросить текущий выбор и начать размещение заново. Блок-схема данного алгоритма представлена на рисунке 3.5.

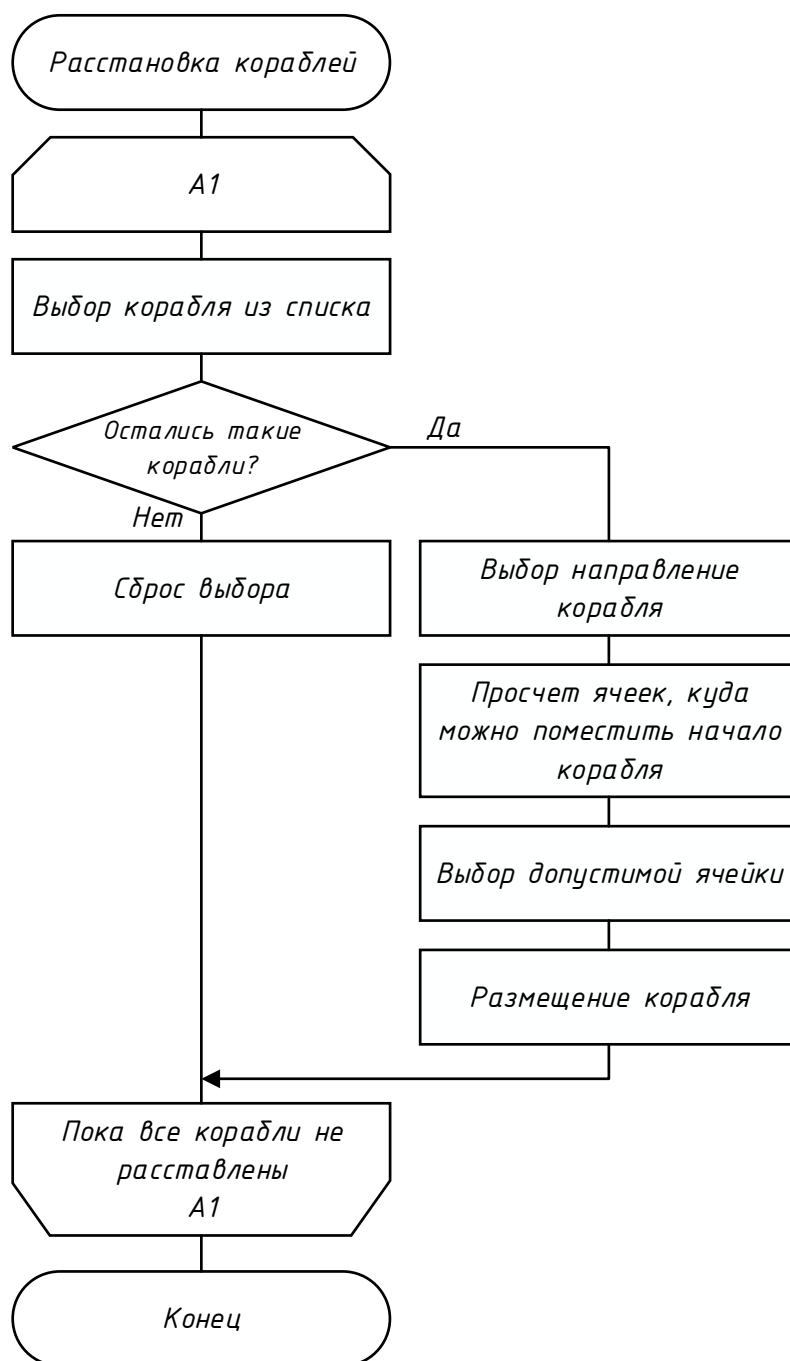


Рисунок 3.5 – Блок-схема алгоритма расстановки кораблей

3.3.2 Авторасстановка кораблей

Алгоритм авторасстановки кораблей работает по следующему принципу: система последовательно выбирает самый большой из оставшихся кораблей и пытается разместить его на поле. Для этого генерируются случайные координаты начальной позиции и направление (горизонтальное или вертикальное). Если размещение невозможно из-за пересечений с другими кораблями или выхода за границы поля, система повторяет попытку с новыми параметрами. После успешного размещения корабль удаляется из списка доступных. Процесс продолжается до полной расстановки всего флота. Блок-схема алгоритма представлена на рисунке 3.7.

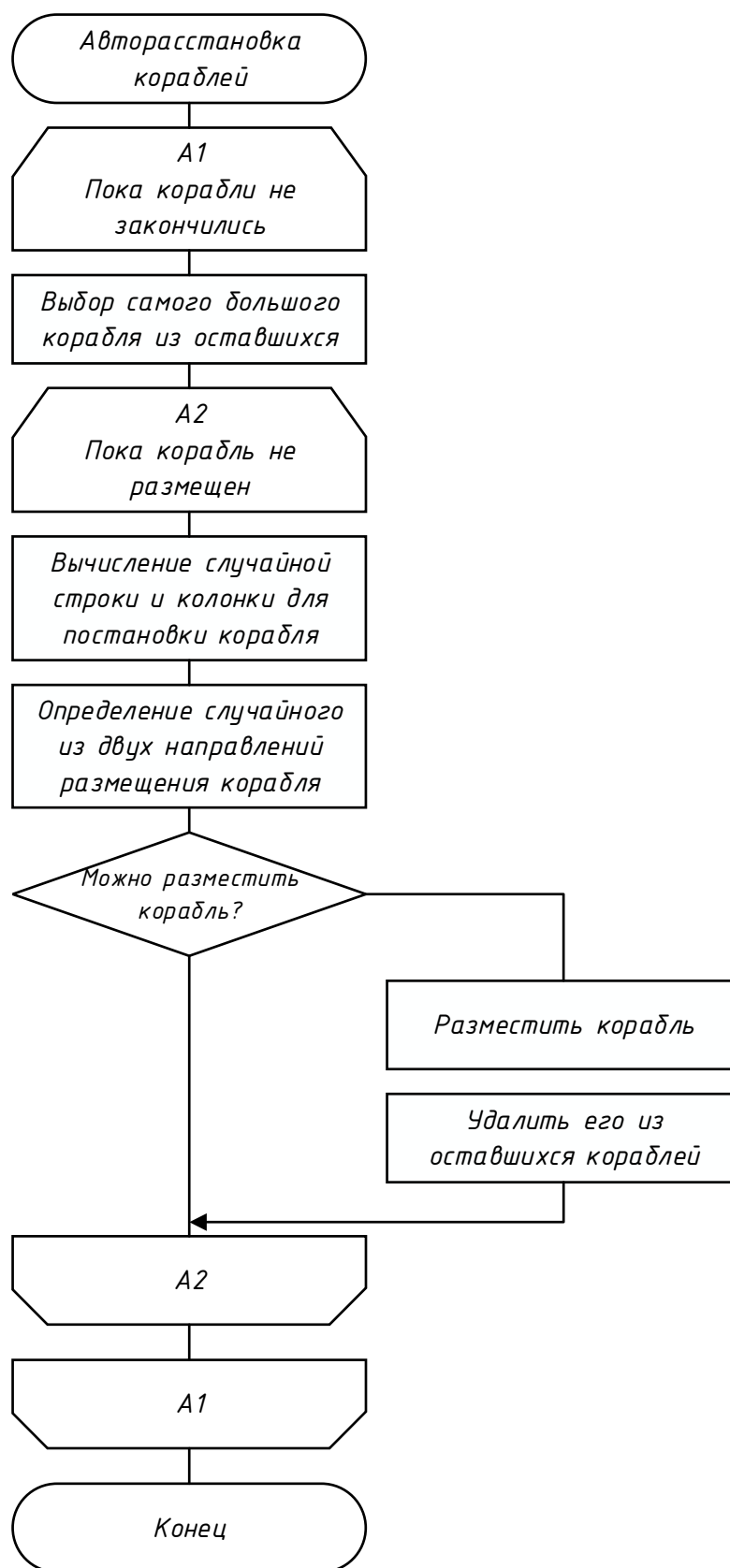


Рисунок 3.6 – Блок-схема алгоритма авторасстановки кораблей

3.3.3 Обработка выстрела

Алгоритм обработки выстрела начинается с проверки состояния целевой ячейки. Сначала система определяет, производился ли ранее выстрел в эту ячейку - если да, ход считается недействительным.

При первом попадании в ячейку происходит анализ её содержимого. Если ячейка пуста, фиксируется промах. При попадании в корабль система отмечает попадание и проверяет, уничтожен ли корабль полностью (все его ячейки поражены). В зависимости от результата ячейка помечается как повреждённая или уничтоженная часть корабля. Процесс завершается обновлением игрового состояния. Блок-схема алгоритма данной функции приведена на рисунке 3.7.

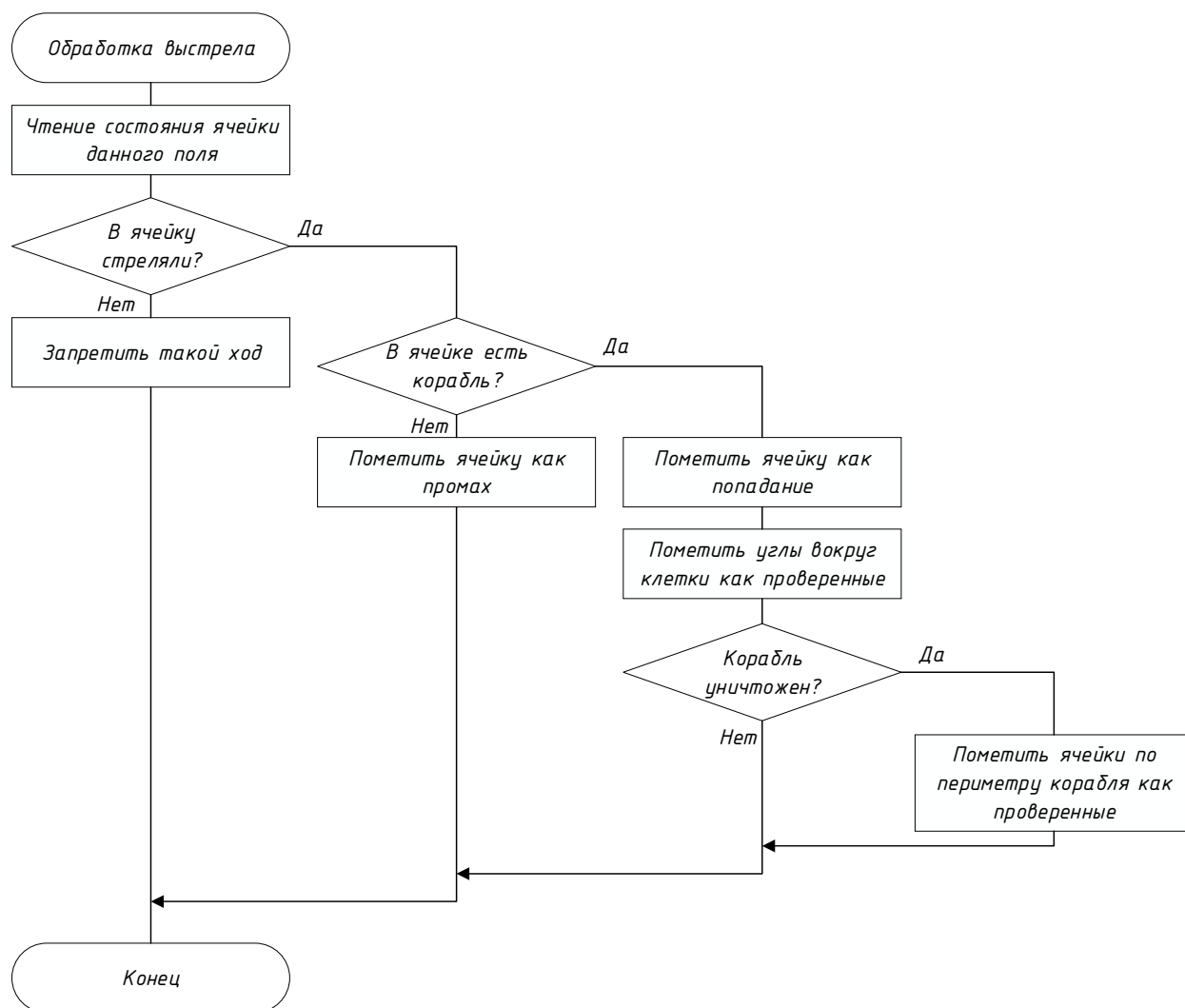


Рисунок 3.7 – Блок-схема алгоритма обработки выстрела

3.3.4 Запуск и управление сетевой игрой

Алгоритм запуска сетевой игры реализует следующий процесс: клиент подключается к серверному хабу и регистрируется в очереди ожидания. Когда набирается два игрока, система создает игровую сессию, генерируя уникальный ID.

Сервер синхронизирует начальные данные - расстановку кораблей и идентификаторы игроков. В течение игрового сеанса сервер принимает данные о ходе от активного игрока и передает их сопернику, обеспечивая поочередность ходов. Процесс продолжается до завершения игры, после чего сервер рассылает финальные результаты участникам. Блок-схема алгоритма данной функции приведена на рисунке 3.8.

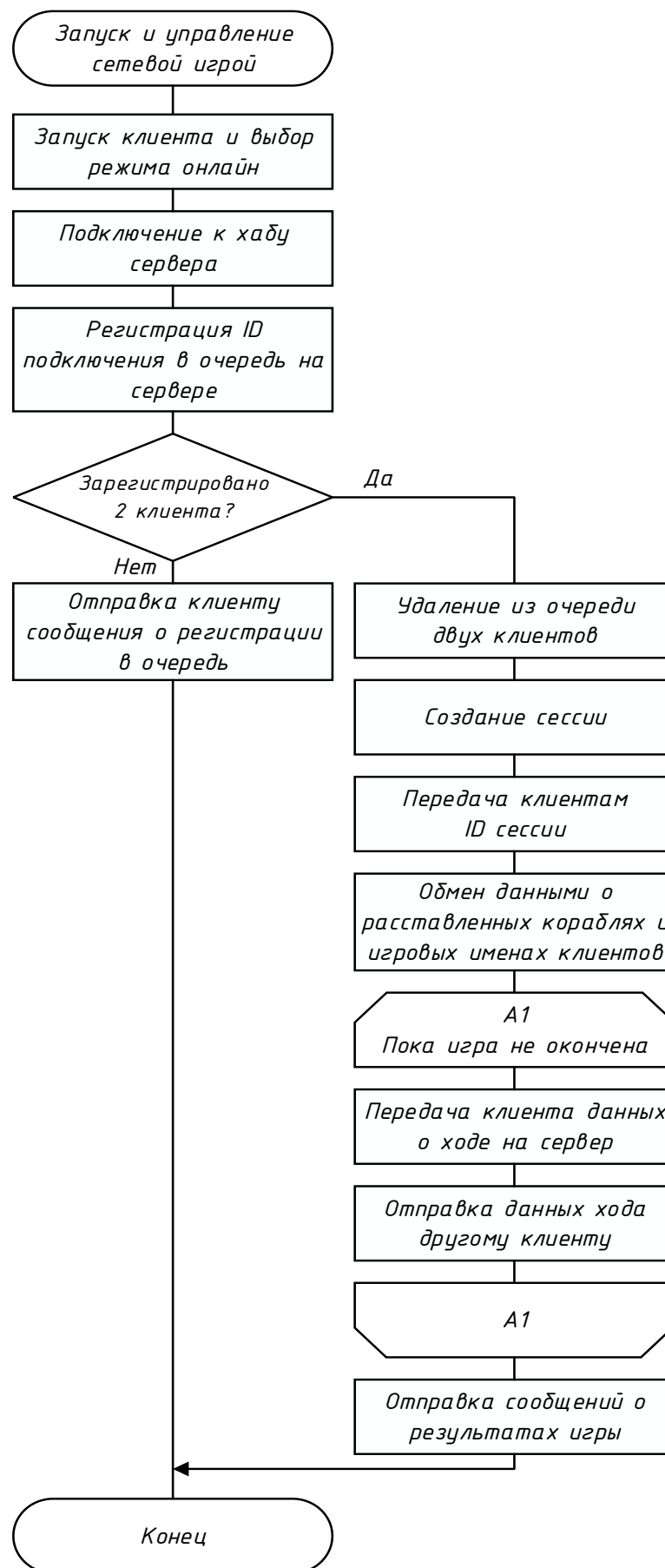


Рисунок 3.8 – Блок-схема алгоритма отображения всех растений

4 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

4.1 Серверная часть: API для онлайн игры

Серверная часть Battleship Online построена на основе SignalR хаба SeaBattleHub, который обрабатывает все аспекты многопользовательской игры в реальном времени. Основной функционал включает управление подключениями игроков, автоматический подбор соперников через систему очередей, синхронизацию игровых состояний и обработку всех игровых событий.

При подключении нового игрока система регистрирует его соединение и добавляет в очередь ожидания. Как только набирается два игрока, автоматически создается новая игровая сессия с уникальным идентификатором. В ходе игры хаб обрабатывает все ключевые события: передачу расстановки кораблей между игроками, обработку ходов, определение попаданий и завершение игры.

```
        public async Task JoinQueue()
    {
        // Добавляем игрока в очередь
        _waitingPlayers.Enqueue(Context.ConnectionId);

        // Уведомляем клиента, что он в очереди
        await Clients.Caller.SendAsync("WaitingForOpponent");

        // Проверяем, можно ли начать игру
        if (_waitingPlayers.Count >= 2)
        {
            // Берем двух игроков из очереди
            if (_waitingPlayers.TryDequeue(out var player1) &&
                _waitingPlayers.TryDequeue(out var player2))
            {
                // Создаем новую игровую сессию
                var sessionId = Guid.NewGuid().ToString();
                var session = new GameSession
                {
                    SessionId = sessionId,
                    Player1ConnectionId = player1,
                    Player2ConnectionId = player2,
                    CurrentTurnPlayerId = player1, // Первый игрок ходит первым
                    IsGameStarted = false
                };

                _gameSessions.TryAdd(sessionId, session);

                // Уведомляем игроков о начале игры
                await Clients.Clients(player1, player2).SendAsync("GameSessionCreated",
sessionId);
            }
        }
    }

    public async Task SendBattlefield(string sessionId, string[][] battlefield)
```

```

{
    if (_gameSessions.TryGetValue(sessionId, out var session))
    {
        // Сохраняем поле игрока и отправляем его сопернику
        if (Context.ConnectionId == session.Player1ConnectionId)
        {
            session.Player1Field = battlefield;
            await Clients.Client(session.Player2ConnectionId)
                .SendAsync("ReceiveOpponentField", session.Player1Field);
        }
        else if (Context.ConnectionId == session.Player2ConnectionId)
        {
            session.Player2Field = battlefield;
            await Clients.Client(session.Player1ConnectionId)
                .SendAsync("ReceiveOpponentField", session.Player2Field);
        }

        // Если оба игрока отправили свои поля, начинаем игру
        if (session.Player1Field != null && session.Player2Field != null && !session.IsGameStarted)
        {
            session.IsGameStarted = true;
            await Clients.Client(session.Player1ConnectionId).SendAsync("GameStarted", true);
            await Clients.Client(session.Player2ConnectionId).SendAsync("GameStarted", false);
        }
    }
}

public async Task MakeMove(string sessionId, PlayerMove move)
{
    if (_gameSessions.TryGetValue(sessionId, out var session))
    {
        // Определяем ID противника
        var opponentId = Context.ConnectionId == session.Player1ConnectionId
            ? session.Player2ConnectionId
            : session.Player1ConnectionId;

        // Передаем ход противнику
        await Clients.Client(opponentId).SendAsync("ReceiveOpponentMove", move);
    }
}

```

Особое внимание уделено надежности соединения - при разрыве связи игрок автоматически удаляется из очереди или текущей сессии, а его соперник получает соответствующее уведомление.

```

public override async Task OnDisconnectedAsync(Exception? exception)
{
    // Удаляем игрока из очереди, если он отключился
    if (_waitingPlayers.Contains(Context.ConnectionId))
    {
        _waitingPlayers.TryDequeue(out _);
    }

    // Завершаем игру, если один из игроков отключился
    var session = _gameSessions.Values.FirstOrDefault(s =>
        s.Player1ConnectionId == Context.ConnectionId ||
        s.Player2ConnectionId == Context.ConnectionId);
}

```

```

    if (session != null)
    {
        _gameSessions.TryRemove(session.SessionId, out _);
        var opponentId = session.Player1ConnectionId == Context.ConnectionId
            ? session.Player2ConnectionId
            : session.Player1ConnectionId;

        if (opponentId != null)
        {
            await Clients.Client(opponentId).SendAsync("OpponentDiscon-
nected");
        }
    }

    await base.OnDisconnectedAsync(exception);
}

```

Дополнительные функции включают синхронизацию имен игроков и поддержку внутриигрового чата для общения между участниками. Все игровые данные хранятся в памяти сервера с использованием потокобезопасных структур, что обеспечивает минимальные задержки при обработке запросов. Архитектура хаба позволяет легко масштабировать систему для поддержки большого количества одновременных игровых сессий.

4.2 Клиентская часть: конструктор поля

Функция `handleCellClick` обрабатывает клик по ячейке игрового поля при расстановке кораблей. Она проверяет, выбран ли корабль и доступен ли он для размещения. Если размещение возможно (с учетом размера, направления и свободного места), функция обновляет матрицу игрового поля, помечая занятые ячейки как "ship", и уменьшает количество доступных кораблей данного типа. После успешного размещения, если кораблей этого типа больше не осталось, сбрасывается текущий выбор корабля.

```

const handleCellClick = useCallback((row: number, col: number) => {
    if (!selectedShip || availableShips[selectedShip.size] <= 0) return;

    const { size, direction } = selectedShip;

    if (SeaFightEngine.canPlaceShip(row, col, size, direction, matrixState)) {
        const newMatrix = [...matrixState];

        // Ставим корабль
        for (let i = 0; i < size; i++) {
            if (direction === 'horizontal') {
                newMatrix[row][col + i] = 'ship';
            } else {
                newMatrix[row + i][col] = 'ship';
            }
        }

        const newAvailableShips = {
            ...availableShips,

```



```

        [size]: availableShips[size] - 1
    };

    setMatrixState(newMatrix);
    setAvailableShips(newAvailableShips);

    // Если кораблей этого типа больше нет - сбрасываем выбор
    if (newAvailableShips[size] <= 0) {
        setSelectedShip(null);
    }
}
}, [selectedShip, matrixState, availableShips]);

```

Функция `canPlaceShip` проверяет, можно ли поставить корабль на поле. Сначала она проверяет, чтобы корабль не выходил за границы поля. Затем убеждается, что все нужные клетки свободны и вокруг них нет других кораблей. Возвращает `true`, если размещение возможно, и `false`, если нет.

```

public static canPlaceShip = (row: number, col: number, size: ShipSize, direction:
ShipDirection, matrix: BattlefieldMatrix): boolean => {
    // Проверяем границы поля
    if (direction === 'horizontal') {
        if (col + size > 10) return false;
    } else {
        if (row + size > 10) return false;
    }

    // Проверяем что все клетки свободны
    for (let i = 0; i < size; i++) {
        const cellRow = direction === 'horizontal' ? row : row + i;
        const cellCol = direction === 'horizontal' ? col + i : col;

        if (matrix[cellRow][cellCol] !== 'empty') return false;

        // Проверяем соседние клетки
        for (let r = Math.max(0, cellRow - 1); r <= Math.min(9, cellRow + 1); r++) {
            for (let c = Math.max(0, cellCol - 1); c <= Math.min(9, cellCol + 1);
c++) {
                if (matrix[r][c] === 'ship') return false;
            }
        }
    }

    return true;
};

```

Функция `generateField` автоматически создает игровое поле для "Морского боя". Она генерирует пустое поле 10x10 и последовательно пытается расставить все корабли (1x4, 2x3, 3x2 и 4x1) в случайных позициях. Для каждого корабля выполняется до 100 попыток найти подходящее место с проверкой через `canPlaceShip`. Если размещение удастся, корабль добавляется на поле. В результате возвращается полностью сгенерированная игровая матрица с корректно расставленными кораблями.

```

public static generateField = () => {
    const newMatrix: BattlefieldMatrix = Array(10).fill(null).map(() => Ar-
ray(10).fill('empty'));

```

```

const shipsToPlace: ShipSize[] = [4, 3, 3, 2, 2, 2, 1, 1, 1, 1];

// Пытаемся расставить каждый корабль
for (const shipSize of shipsToPlace) {
  let placed = false;
  let attempts = 0;

  while (!placed && attempts < 100) {
    attempts++;
    const direction: ShipDirection = Math.random() > 0.5 ? 'horizontal' :
'vertical';
    const row = Math.floor(Math.random() * 10);
    const col = Math.floor(Math.random() * 10);

    if (SeaFightEngine.canPlaceShip(row, col, shipSize, direction, new-
Matrix)) {
      // Ставим корабль
      for (let i = 0; i < shipSize; i++) {
        if (direction === 'horizontal') {
          newMatrix[row][col + i] = 'ship';
        } else {
          newMatrix[row + i][col] = 'ship';
        }
      }
      placed = true;
    }
  }
}

return newMatrix;
}

```

4.3 Клиентская часть: игровой процесс

Хук `useSeaBattleHub` реализует клиентскую часть сетевого взаимодействия для игры "Морской бой" через SignalR. Он устанавливает соединение с игровым сервером и обрабатывает все основные события: подключение к игре, синхронизацию игровых состояний, обработку ходов и чат. При инициализации хук создает SignalR-соединение и автоматически подключается к очереди игроков. После успешного соединения регистрируются обработчики всех возможных серверных событий - от создания игровой сессии до получения сообщений чата.

Основные функции включают отправку игрового поля на сервер, обработку ходов игроков, управление чатом и передачу имен участников. Хук также отслеживает состояние игры (текущая сессия, очередь хода, сообщения чата) и предоставляет его через возвращаемый объект.

```

export const useSeaBattleHub = (
  onGameStarted: (isPlayerTurn: boolean) => void,
  setOpponentField: (field: BattlefieldMatrix) => void,
  setGameStatus: (status: GameStatus) => void,
) => {
  const [connection, setConnection] = useState<signalR.HubConnection |
null>(null);

```

```

    const [session, setSession] = useState<GameSession | null>(null);
    const [chatMessages, setChatMessages] = useState<{sender: string, message:
string}[]>([]);
    const [playerName, setPlayerName] = useState<string>('');

    // Устанавливаем соединение с хабом на сервере
    useEffect(() => {
        const newConnection = new signalR.HubConnectionBuilder()
            .withUrl(`${BASE_API_URL}/seabattlehub`)
            .withAutomaticReconnect()
            .build();

        setConnection(newConnection);

        return () => {
            newConnection.stop();
        };
    }, [connection]);

    // Когда изменился connection стартуем соединение и вызываем joinQueue();
    useEffect(() => {
        if (connection) {
            connection.start()
                .then(() => {
                    setupListeners();
                    joinQueue();
                })
                .catch(err => console.error('Connection failed: ', err));
        }
    }, [connection]);

    const setupListeners = () => {
        if (!connection) return;

        connection.on('WaitingForOpponent', () => {
            console.log('Waiting for opponent...');
        });

        connection.on('GameSessionCreated', (sessionId: string) => {
            setSession(prev => ({
                ...prev,
                sessionId,
                isPlayerTurn: false
            }));
            console.log('Game session created... Session id =', sessionId);
        });

        connection.on('GameStarted', (isPlayerTurn: boolean) => {
            setSession(prev => prev ? {
                ...prev,
                isPlayerTurn
            } : null);
            console.log(`Game started...`);
            onGameStarted(isPlayerTurn);
        });

        connection.on('ReceiveOpponentField', (battlefield: string[][]) => {
            if (setOpponentField) {
                setOpponentField(battlefield as BattlefieldMatrix);
                console.log(`Opponent field received...`);
            }
        });
    };

```

```

    });

    connection.on('OpponentDisconnected', () => {
        console.log('Opponent disconnected');
        setSession(null);
        setGameStatus('opponentLeave');
    });

    connection.on('ReceiveChatMessage', (sender: string, message: string)
=> {
        setChatMessages(prev => [...prev, {sender, message}]);
    });

    connection.on('ReceiveOpponentName', (name: string) => {
        setSession(prev => prev ? {
            ...prev,
            opponentName: name
        } : null);
    });
};

const joinQueue = useCallback(() => {
    if (connection) {
        connection.invoke('JoinQueue')
            .catch(err => console.error('JoinQueue failed: ', err));
    }
}, [connection]);

const sendBattlefield = useCallback((sessionId: string, battlefield:
string[][]) => {
    if (connection) {
        connection.invoke('SendBattlefield', sessionId, battlefield)
            .catch(err => console.error('SendBattlefield failed: ', err));
    }
}, [connection]);

const makeMove = useCallback((sessionId: string, row: number, col: number)
=> {
    if (connection) {
        connection.invoke('MakeMove', sessionId, { row, column: col })
            .catch(err => console.error('MakeMove failed: ', err));
    }
}, [connection]);

const sendChatMessage = useCallback((sessionId: string, message: string) =>
{
    if (connection && playerName) {
        connection.invoke('SendChatMessage', sessionId, playerName, mes-
sage)
            .catch(err => console.error('SendChatMessage failed: ', err));
    }
    setChatMessages(prev => [...prev, {sender: playerName, message}]);
}, [connection, playerName]);

const sendPlayerName = useCallback((sessionId: string, name: string) => {
    if (connection) {
        setPlayerName(name);
        connection.invoke('SendPlayerName', sessionId, name)
            .then(() => {
                setSession(prev => prev ? {
                    ...prev,

```

```

        playerName: name
      } : null);
    })
    .catch(err => console.error('SendPlayerName failed: ', err));
  }
}, [connection]);

return {
  session,
  connection,
  chatMessages,
  sendBattlefield,
  makeMove,
  sendChatMessage,
  sendPlayerName
};
};

```

При разрыве соединения или отключении соперника хук автоматически обновляет игровой статус. Вся сетевая логика инкапсулирована в этом хуке, что позволяет компонентам игры работать только с высокоуровневыми методами без необходимости прямого взаимодействия с SignalR API.

5 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

5.1 Запуск сервера и подключение клиента

Исходные данные: запуск сервера, открытие клиента в браузере в онлайн-режиме.

Ожидаемый результат: отображение сообщения об ожидании соперника.

Тест пройден успешно.

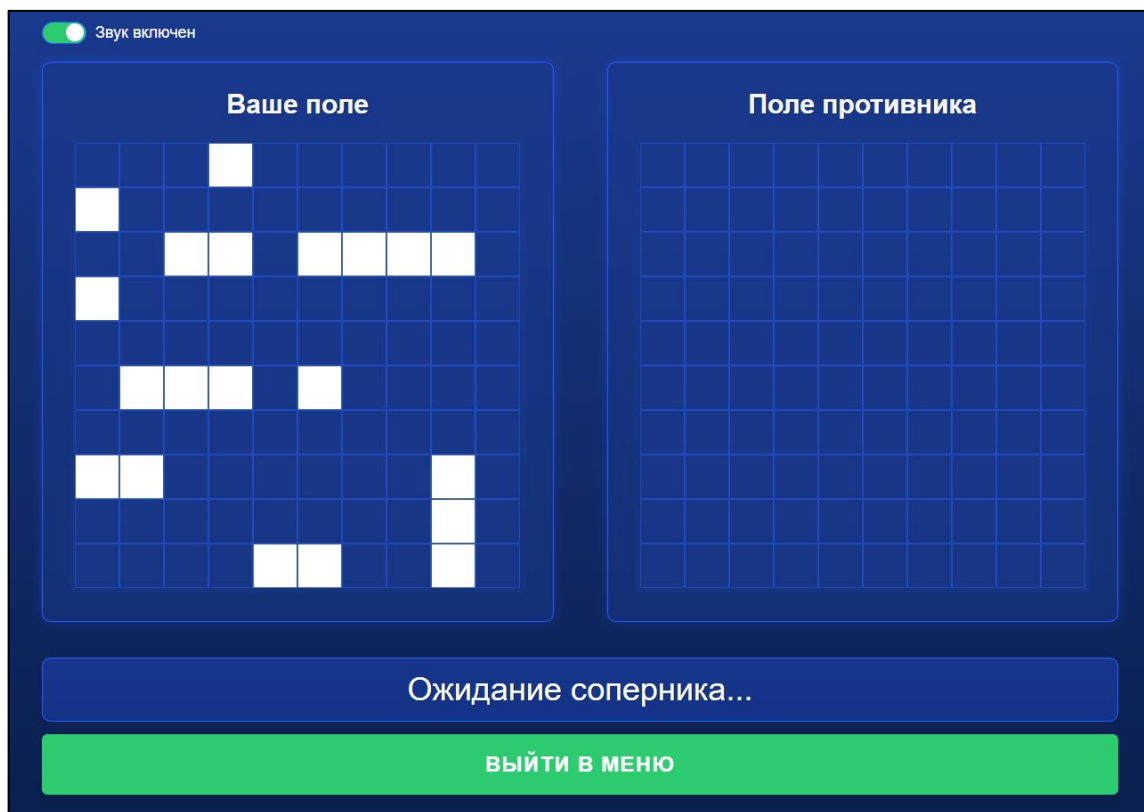


Рисунок 5.1 – Запуск и подключение клиента

5.2 Игровой процесс

Исходные данные: осуществление нескольких ходов.

Ожидаемый результат: изменение состояния полей у игрока и его соперника.

Тест пройден успешно.

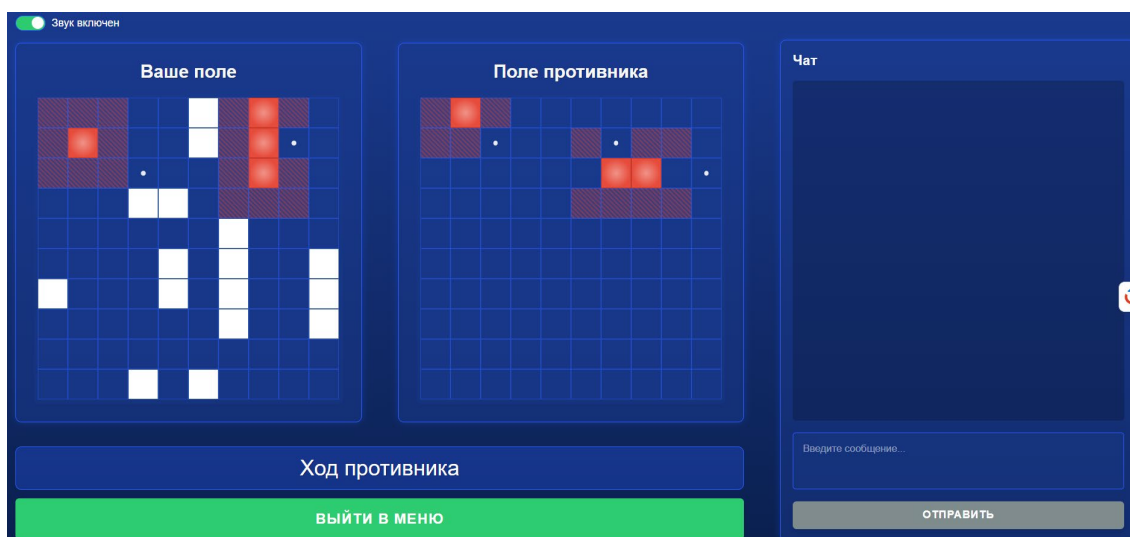


Рисунок 5.2 – Игровой процесс в режиме онлайн

5.3 Чат

Исходные данные: отправка нескольких сообщений.

Ожидаемый результат: отображение сообщений в чатах обоих клиентов.

Тест пройден успешно.

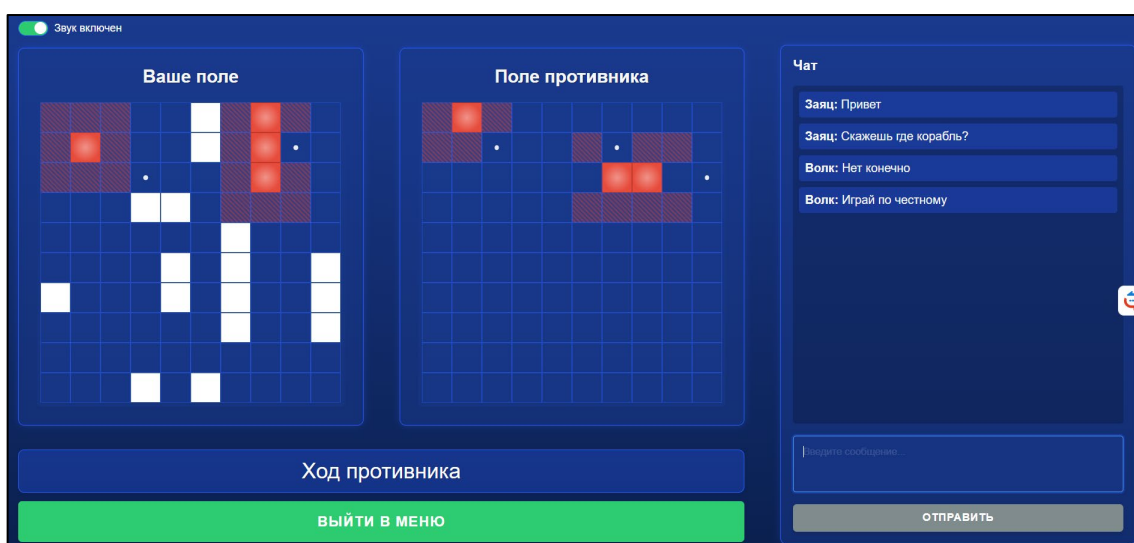


Рисунок 5.3 – Чат в режиме онлайн

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

6.1 Начало работы в «Battleship Online»

Игра «Морской Бой» доступна в браузере и не требует установки дополнительного программного обеспечения. Главный экран предлагает три основных варианта: начать сетевую игру с реальным соперником, сразиться с компьютером или ознакомиться с правилами игры. Новые игроки могут сразу приступить к бою, выбрав один из режимов с ботом, а для сетевой игры потребуется указать своё имя. Интерфейс выполнен в классическом морском стиле с интуитивно понятным управлением.

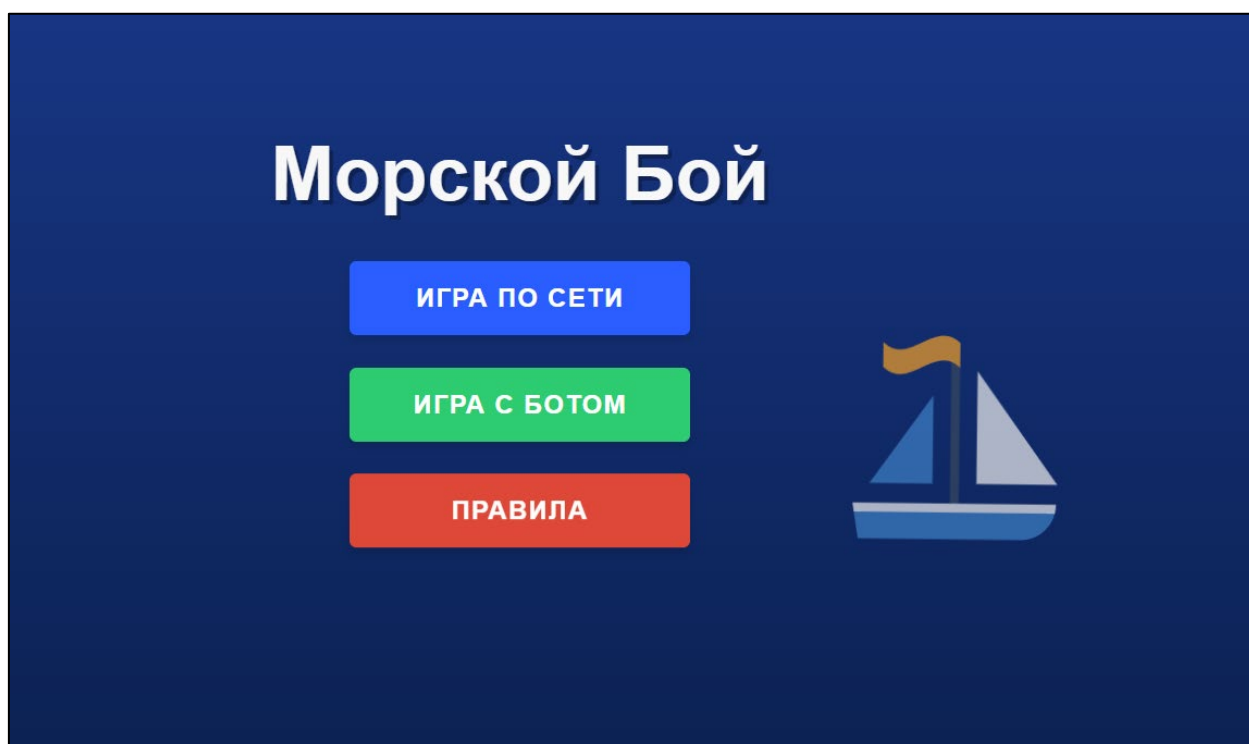


Рисунок 6.1 – Главная страница

6.2 Страница расстановки кораблей

Страница расстановки кораблей представляет собой основной интерфейс для подготовки к игре. В верхней части экрана отображается поле для ввода игрового псевдонима. Центральную зону занимает панель с информацией о доступных кораблях, где показано количество оставшихся для размещения кораблей каждого типа.

Под панелью кораблей расположены элементы управления: кнопка поворота корабля (горизонтально/вертикально), функция авторасстановки и опция сброса текущей расстановки. Основное пространство экрана отведено

под игровое поле 10×10 клеток, где происходит непосредственное размещение кораблей.

В нижней части экрана находятся две основные кнопки действий: возврат в главное меню и начало игры. Кнопка "Начать игру" активируется только после корректной расстановки всех кораблей согласно правилам. Интерфейс выполнен в морской тематике с использованием сине-голубой цветовой гаммы, что соответствует атмосфере классической игры "Морской бой".

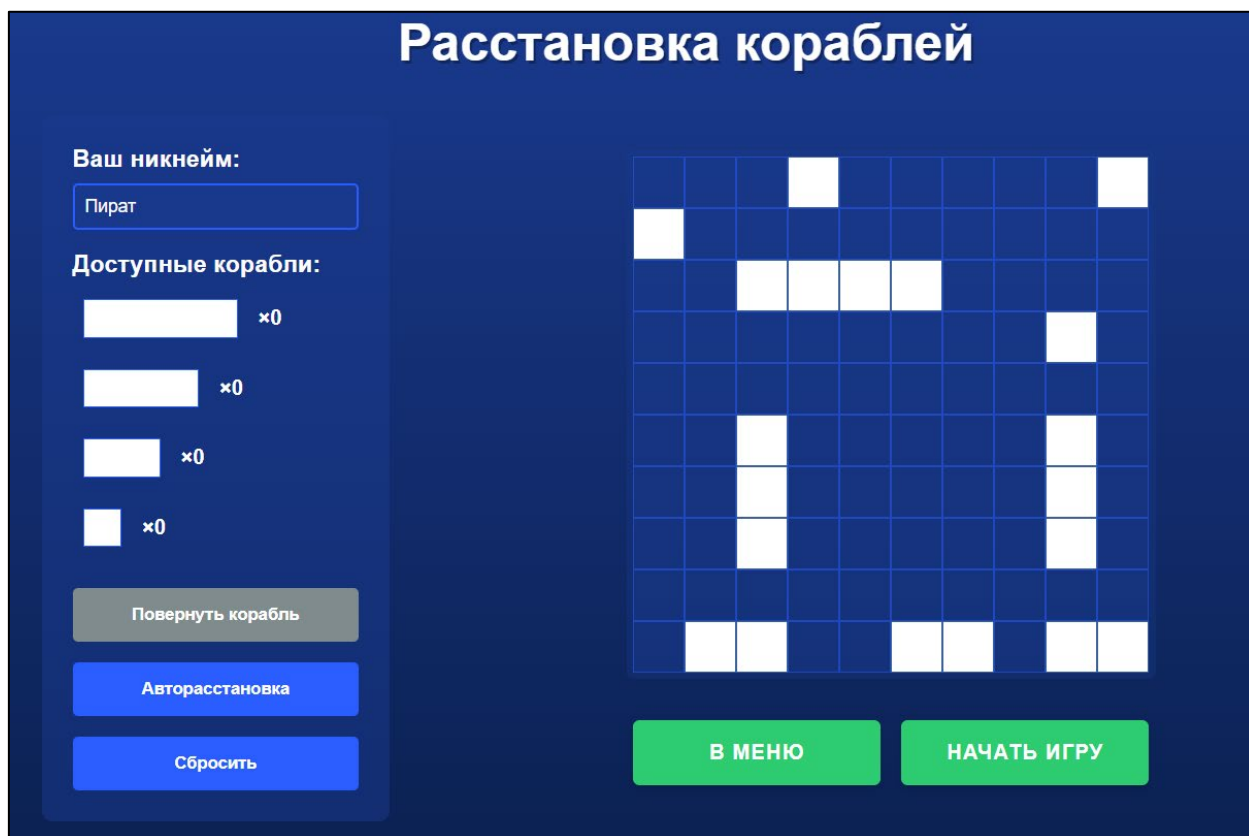


Рисунок 6.2 – Страница расстановки кораблей

6.3 Страница игрового процесса

Экран игрового процесса разделен на две основные части. В левой половине отображается собственное поле игрока с текущей расстановкой кораблей и отметками о попаданиях противника. В правой части находится поле соперника, где игрок отмечает свои атаки. В верхней части интерфейса отображается статус текущего хода, напоминающий, чья сейчас очередь атаковать.

В правой части экрана расположен игровой чат, позволяющий обмениваться сообщениями с оппонентом. Поле ввода сообщений и кнопка отправки обеспечивают удобную коммуникацию во время боя. В углу

интерфейса находится кнопка выхода в главное меню, позволяющая прервать текущую игру при необходимости.

Все элементы интерфейса выполнены в едином морском стиле с использованием синих оттенков и тематических графических элементов.

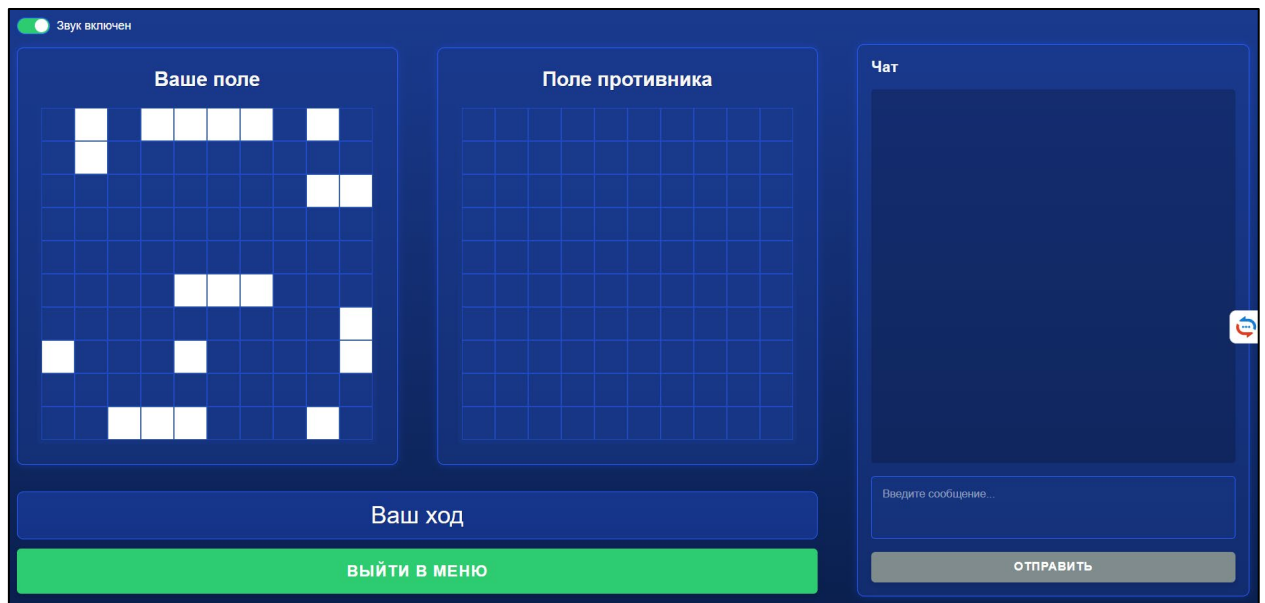


Рисунок 6.3 – Страница игрового процесса

6.4 Правила игры

Окно правил кратко объясняет основы игры: два игрока по очереди стреляют по полю 10×10 , пытаясь первыми потопить все корабли противника. Перечислены типы кораблей (от 1-клеточных до 4-клеточных) и главное правило расстановки — корабли не должны соприкасаться. Внизу есть кнопка "Понятно" для закрытия окна.

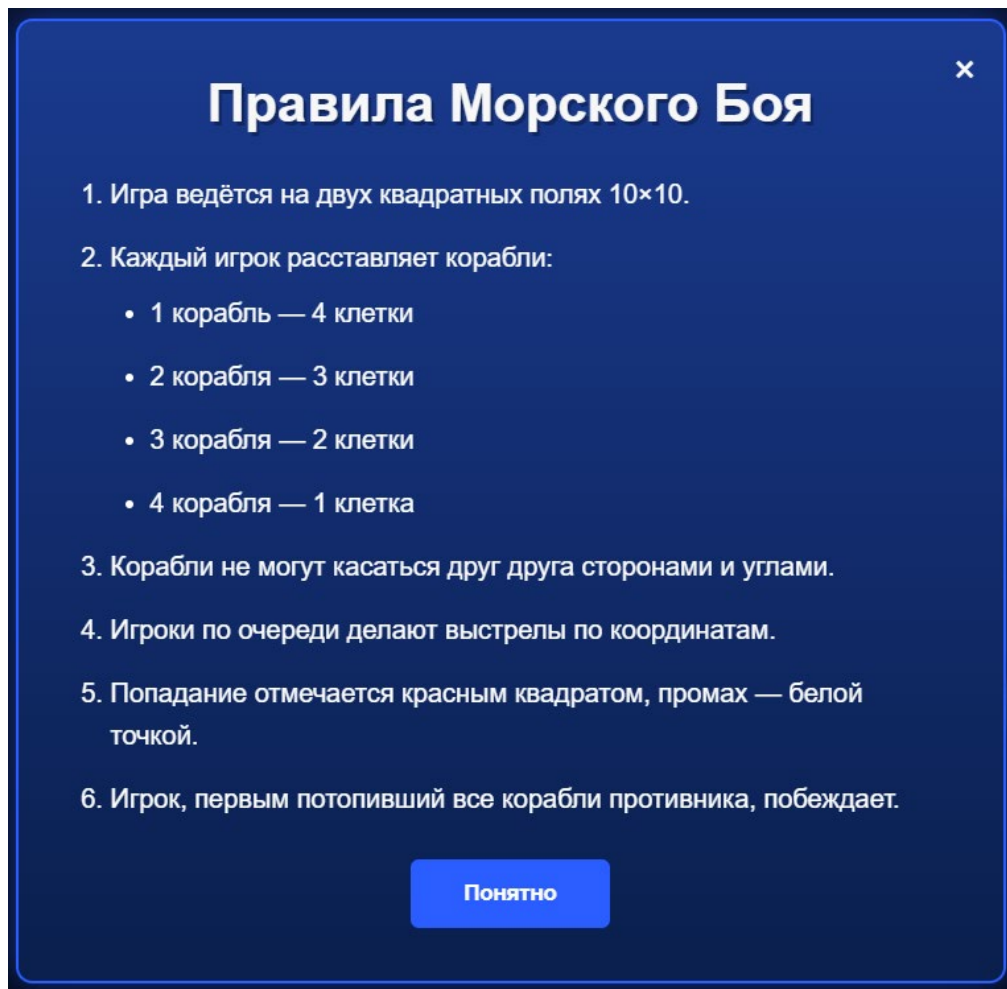


Рисунок 6.4 – Правила игры

ЗАКЛЮЧЕНИЕ

"Battleship Online" представляет собой современную цифровую версию классической игры "Морской бой", сохранившую свою популярность среди игроков всех возрастов. В ходе реализации проекта были успешно решены все ключевые задачи:

- разработан интуитивно понятный интерфейс расстановки кораблей;
- реализована система автоматической генерации игровых полей;
- создан интеллектуальный алгоритм игры против компьютера;
- обеспечена наглядная визуализация игрового процесса;
- внедрена система сетевой игры через интернет.

Для достижения этих результатов потребовалось глубокое изучение технологий веб-разработки, включая React, TypeScript и SignalR для реализации многопользовательского режима.

Проект обладает значительным потенциалом для дальнейшего развития. Среди возможных улучшений - внедрение системы рейтингов игроков, добавление различных режимов игры (турниры, командные сражения), а также расширение функционала чата.

"Battleship Online" предлагает пользователям удобную платформу для ностальгического возвращения в детство или знакомства с классической игрой в современном цифровом формате, сочетая простые правила с увлекательным игровым процессом.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

- [1] Официальная документация React [Электронный ресурс]. – Режим доступа: <https://react.dev/> – Дата доступа: 10.05.2025.
- [2] Официальная документация TypeScript [Электронный ресурс]. – Режим доступа: <https://www.typescriptlang.org/docs/> – Дата доступа: 12.05.2025.
- [3] ASP.NET Core Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/en-us/aspnet/core/> – Дата доступа: 15.05.2025.
- [4] "Building Scalable Web Applications with ASP.NET Core" / R. Anderson. – O'Reilly, 2023. – Режим доступа: <https://www.oreilly.com/library/view/building-scalable-web/9781098102271/> – Дата доступа: 20.05.2025.
- [5] "React and TypeScript: Build Scalable Apps" / C. Nwamba. – Apress, 2022. – Режим доступа: <https://link.springer.com/book/10.1007/978-1-4842-7994-0> – Дата доступа: 22.05.2025.
- [6] MDN Web Docs: HTTP Protocol [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/HTTP> – Дата доступа: 22.05.2025.
- [7] "Clean Code in TypeScript" / J. Boyarsky. – Manning Publications, 2024. – Режим доступа: <https://www.manning.com/books/clean-code-in-typescript> – Дата доступа: 22.05.2025.
- [8] Vite Official Documentation [Электронный ресурс]. – Режим доступа: <https://vitejs.dev/guide/> – Дата доступа: 22.05.2025.

ПРИЛОЖЕНИЕ А

Исходный код программы

Реализация хаба на сервере

```
using Microsoft.AspNetCore.SignalR;
using SeaFightServer.Models;
using System.Collections.Concurrent;

namespace SeaFightServer.Hubs
{
    public class SeaBattleHub : Hub
    {
        private static readonly ConcurrentQueue<string> _waitingPlayers = new
ConcurrentQueue<string>();
        private static readonly ConcurrentDictionary<string, GameSession>
_gameSessions = new ConcurrentDictionary<string, GameSession>();

        public override async Task OnConnectedAsync()
        {
            Console.WriteLine($"Client connected: {Context.ConnectionId}");
            await base.OnConnectedAsync();
        }

        public override async Task OnDisconnectedAsync(Exception? exception)
        {
            // Удаляем игрока из очереди, если он отключился
            if (_waitingPlayers.Contains(Context.ConnectionId))
            {
                _waitingPlayers.TryDequeue(out _);
            }

            // Завершаем игру, если один из игроков отключился
            var session = _gameSessions.Values.FirstOrDefault(s =>
                s.Player1ConnectionId == Context.ConnectionId ||
                s.Player2ConnectionId == Context.ConnectionId);

            if (session != null)
            {
                _gameSessions.TryRemove(session.SessionId, out _);
                var opponentId = session.Player1ConnectionId ==
Context.ConnectionId
                    ? session.Player2ConnectionId
                    : session.Player1ConnectionId;

                if (opponentId != null)
                {
                    await
Clients.Client(opponentId).SendAsync("OpponentDisconnected");
                }
            }

            await base.OnDisconnectedAsync(exception);
        }
    }
}
```

```

public async Task JoinQueue()
{
    // Добавляем игрока в очередь
    _waitingPlayers.Enqueue(Context.ConnectionId);

    // Уведомляем клиента, что он в очереди
    await Clients.Caller.SendAsync("WaitingForOpponent");

    // Проверяем, можно ли начать игру
    if (_waitingPlayers.Count >= 2)
    {
        // Берем двух игроков из очереди
        if (_waitingPlayers.TryDequeue(out var player1) &&
            _waitingPlayers.TryDequeue(out var player2))
        {
            // Создаем новую игровую сессию
            var sessionId = Guid.NewGuid().ToString();
            var session = new GameSession
            {
                SessionId = sessionId,
                Player1ConnectionId = player1,
                Player2ConnectionId = player2,
                CurrentTurnPlayerId = player1, // Первый игрок ходит
                IsGameStarted = false
            };

            _gameSessions.TryAdd(sessionId, session);

            // Уведомляем игроков о начале игры
            await Clients.Clients(player1,
player2).SendAsync("GameSessionCreated", sessionId);
        }
    }

    public async Task SendBattlefield(string sessionId, string[][]
battlefield)
    {
        if (_gameSessions.TryGetValue(sessionId, out var session))
        {
            // Сохраняем поле игрока и отправляем его сопернику
            if (Context.ConnectionId == session.Player1ConnectionId)
            {
                session.Player1Field = battlefield;
                await Clients.Client(session.Player2ConnectionId)
                    .SendAsync("ReceiveOpponentField",
session.Player1Field);
            }
            else if (Context.ConnectionId == session.Player2ConnectionId)
            {
                session.Player2Field = battlefield;
                await Clients.Client(session.Player1ConnectionId)

```

```

        .SendAsync("ReceiveOpponentField",
session.Player2Field);
    }

    // Если оба игрока отправили свои поля, начинаем игру
    if (session.Player1Field != null && session.Player2Field !=
null && !session.IsGameStarted)
    {
        session.IsGameStarted = true;
        await
Clients.Client(session.Player1ConnectionId).SendAsync("GameStarted", true);
        await
Clients.Client(session.Player2ConnectionId).SendAsync("GameStarted", false);
    }
}

public async Task MakeMove(string sessionId, PlayerMove move)
{
    if (_gameSessions.TryGetValue(sessionId, out var session))
    {
        // Определяем ID противника
        var opponentId = Context.ConnectionId ==
session.Player1ConnectionId
            ? session.Player2ConnectionId
            : session.Player1ConnectionId;

        // Передаем ход противнику
        await
Clients.Client(opponentId).SendAsync("ReceiveOpponentMove", move);
    }
}

public async Task SendPlayerName(string sessionId, string name)
{
    if (_gameSessions.TryGetValue(sessionId, out var session))
    {
        // Сохраняем имя игрока и отправляем его сопернику
        if (Context.ConnectionId == session.Player1ConnectionId)
        {
            session.Player1Name = name;
            await Clients.Client(session.Player2ConnectionId)
                .SendAsync("ReceiveOpponentName", name);
        }
        else if (Context.ConnectionId == session.Player2ConnectionId)
        {
            session.Player2Name = name;
            await Clients.Client(session.Player1ConnectionId)
                .SendAsync("ReceiveOpponentName", name);
        }
    }
}

public async Task SendChatMessage(string sessionId, string
senderName, string message)

```



```

    {
        if (_gameSessions.TryGetValue(sessionId, out var session))
        {
            // Определяем ID противника
            var opponentId = Context.ConnectionId ==
session.Player1ConnectionId
                ? session.Player2ConnectionId
                : session.Player1ConnectionId;

            // Передаем сообщение противнику
            await Clients.Client(opponentId)
                .SendAsync("ReceiveChatMessage", senderName, message);
        }
    }
}

```

Реализация хука для работы с хабом

```

import { useCallback, useEffect, useState } from 'react';
import * as signalR from '@microsoft/signalr';
import { BASE_API_URL } from "@/configures/server.configure.ts";
import type { BattlefieldMatrix } from "@/types/ship.types.ts";
import type { GameStatus } from "@/types/game.types.ts";

interface GameSession {
    sessionId: string;
    isPlayerTurn: boolean;
    playerName?: string;
    opponentName?: string;
}

export const useSeaBattleHub = (
    onGameStarted: (isPlayerTurn: boolean) => void,
    setOpponentField: (field: BattlefieldMatrix) => void,
    setGameStatus: (status: GameStatus) => void,
) => {
    const [connection, setConnection] = useState<signalR.HubConnection | null>(null);
    const [session, setSession] = useState<GameSession | null>(null);
    const [chatMessages, setChatMessages] = useState<{sender: string, message: string}[]>([]);
    const [playerName, setPlayerName] = useState<string>('');

    // Устанавливаем соединение с хабом на сервере
    useEffect(() => {
        const newConnection = new signalR.HubConnectionBuilder()
            .withUrl(`${BASE_API_URL}/seabattlehub`)
            .withAutomaticReconnect()
            .build();

        setConnection(newConnection);

        return () => {
            newConnection.stop();
        }
    }, []);
}

```

```

    });
  }, []);

  // Когда изменился connection стартуем соединение и вызываем joinQueue();
  useEffect(() => {
    if (connection) {
      connection.start()
        .then(() => {
          setupListeners();
          joinQueue();
        })
        .catch(err => console.error('Connection failed: ', err));
    }
  }, [connection]);

  const setupListeners = () => {
    if (!connection) return;

    connection.on('WaitingForOpponent', () => {
      console.log('Waiting for opponent...');
    });

    connection.on('GameSessionCreated', (sessionId: string) => {
      setSession(prev => ({
        ...prev,
        sessionId,
        isPlayerTurn: false
      }));
      console.log('Game session created... Session id =', sessionId);
    });

    connection.on('GameStarted', (isPlayerTurn: boolean) => {
      setSession(prev => prev ? {
        ...prev,
        isPlayerTurn
      } : null);
      console.log(`Game started...`);
      onGameStarted(isPlayerTurn);
    });

    connection.on('ReceiveOpponentField', (battlefield: string[][]) => {
      if (setOpponentField) {
        setOpponentField(battlefield as BattlefieldMatrix);
        console.log(`Opponent field received...`);
      }
    });

    connection.on('OpponentDisconnected', () => {
      console.log('Opponent disconnected');
      setSession(null);
      setGameStatus('opponentLeave');
    });

    connection.on('ReceiveChatMessage', (sender: string, message: string)
=> {

```

```

        setChatMessages(prev => [...prev, {sender, message}]);
    });

    connection.on('ReceiveOpponentName', (name: string) => {
        setSession(prev => prev ? {
            ...prev,
            opponentName: name
        } : null);
    });
};

const joinQueue = useCallback(() => {
    if (connection) {
        connection.invoke('JoinQueue')
            .catch(err => console.error('JoinQueue failed: ', err));
    }
}, [connection]);

const sendBattlefield = useCallback((sessionId: string, battlefield:
string[][]) => {
    if (connection) {
        connection.invoke('SendBattlefield', sessionId, battlefield)
            .catch(err => console.error('SendBattlefield failed: ',
err));
    }
}, [connection]);

const makeMove = useCallback((sessionId: string, row: number, col:
number) => {
    if (connection) {
        connection.invoke('MakeMove', sessionId, { row, column: col })
            .catch(err => console.error('MakeMove failed: ', err));
    }
}, [connection]);

const sendChatMessage = useCallback((sessionId: string, message: string)
=> {
    if (connection && playerName) {
        connection.invoke('SendChatMessage', sessionId, playerName,
message)
            .catch(err => console.error('SendChatMessage failed: ',
err));
    }
    setChatMessages(prev => [...prev, {sender: playerName, message}]);
}, [connection, playerName]);

const sendPlayerName = useCallback((sessionId: string, name: string) => {
    if (connection) {
        setPlayerName(name);
        connection.invoke('SendPlayerName', sessionId, name)
            .then(() => {
                setSession(prev => prev ? {
                    ...prev,
                    playerName: name
                } : null);
            });
    }
}, [connection]);

```

```

        })
        .catch(err => console.error('SendPlayerName failed: ', err));
    }
  }, [connection]);

  return {
    session,
    connection,
    chatMessages,
    sendBattlefield,
    makeMove,
    sendChatMessage,
    sendPlayerName
  };
};

```

Модуль для расстановки кораблей

```

import React, { useState, useCallback } from 'react';
import styles from './styles/SetupModule.module.css';
import type {BattlefieldMatrix, ShipDirection, ShipSize} from
"@/types/ship.types.ts";
import {Battlefield} from "@/components/battlefield";
import {Button} from "@/ui/button";
import {SeaFightEngine} from "@/engines/seaFightEngine.ts";
import {useNavigate} from "react-router-dom";

interface SetupModuleProps {
  onStartGame: (userField: BattlefieldMatrix, userName: string) => void;
}

const SetupModule = ({onStartGame} : SetupModuleProps) => {
  const [matrixState, setMatrixState] = useState<BattlefieldMatrix>(
    Array(10).fill(null).map(() => Array(10).fill('empty'))
  );
  const [selectedShip, setSelectedShip] = useState<{
    size: ShipSize;
    direction: ShipDirection;
  } | null>(null);
  const [availableShips, setAvailableShips] = useState<Record<ShipSize,
number>>({
    1: 4,
    2: 3,
    3: 2,
    4: 1
  }));
  const [playerName, setPlayerName] = useState('');

  const handleCellClick = useCallback((row: number, col: number) => {
    if (!selectedShip || availableShips[selectedShip.size] <= 0) return;

    const { size, direction } = selectedShip;

    if (SeaFightEngine.canPlaceShip(row, col, size, direction,
matrixState)) {
      const newMatrix = [...matrixState];

```

```

    // Ставим корабль
    for (let i = 0; i < size; i++) {
      if (direction === 'horizontal') {
        newMatrix[row][col + i] = 'ship';
      } else {
        newMatrix[row + i][col] = 'ship';
      }
    }

    const newAvailableShips = {
      ...availableShips,
      [size]: availableShips[size] - 1
    };

    setMatrixState(newMatrix);
    setAvailableShips(newAvailableShips);

    // Если кораблей этого типа больше нет - сбрасываем выбор
    if (newAvailableShips[size] <= 0) {
      setSelectedShip(null);
    }
  }, [selectedShip, matrixState, availableShips]);
  const autoPlaceAllShips = useCallback(() => {
    const newMatrix = SeaFightEngine.generateField();
    setMatrixState(newMatrix);
    setAvailableShips({ 1: 0, 2: 0, 3: 0, 4: 0 });
    setSelectedShip(null);
  }, []);
  const resetField = useCallback(() => {
    setMatrixState(Array(10).fill(null).map(() =>
Array(10).fill('empty')));
    setAvailableShips({ 1: 4, 2: 3, 3: 2, 4: 1 });
    setSelectedShip(null);
  }, []);
  const handleShipSelect = useCallback((size: ShipSize) => {
    if (availableShips[size] <= 0) return;

    setSelectedShip({
      size,
      direction: 'horizontal' // Всегда горизонтальный в меню
    });
  }, [availableShips]);
  const toggleShipDirection = useCallback(() => {
    if (selectedShip) {
      setSelectedShip({
        ...selectedShip,
        direction: selectedShip.direction === 'horizontal' ?
'vertical' : 'horizontal'
      });
    }
  }, [selectedShip]);
  const handleNameChange = useCallback((e:
React.ChangeEvent<HTMLInputElement>) => {

```

```

    setPlayerName(e.target.value);
  }, []);
  const handleStartGame = useCallback(() => {
    const name = playerName.trim() || 'Игрок';
    onStartGame(matrixState, name);
  }, [playerName, matrixState, onStartGame]);
  const navigate = useNavigate();

  return (
    <div className={styles['setup-page']}>
      <h1 className={styles['title']}>Расстановка кораблей</h1>

      <div className={styles['container']}>
        <div className={styles['ships-panel']}>
          <div className={styles['registration-panel']}>
            <h3>Ваш никнейм:</h3>
            <input
              type="text"
              value={playerName}
              onChange={handleNameChange}
              placeholder="Введите ваш ник"
              maxLength={30}
              className={styles['name-input']}
            />
          </div>

          <h3>Доступные корабли:</h3>
          <div className={styles['ships']}>
            {[4, 3, 2, 1].map((size) => (
              <div
                key={size}
                className={` ${styles['ship-item']} ${
styles['ship-item--selected'] : ''
                }`}
                onClick={() => handleShipSelect(size as
ShipSize)}
              >
                <div
                  className={styles['ship-preview']}
                  data-size={size}
                />
                <span className={styles['ship-
count']}>×{availableShips[size as ShipSize]}</span>
              </div>
            )]}
          </div>

          <button
            className={styles['rotate-button']}
            onClick={toggleShipDirection}
            disabled={!selectedShip}
          >
            Повернуть корабль
          </button>

```

```

    <div className={styles.buttons}>
      <button
        className={styles.button}
        onClick={autoPlaceAllShips}
      >
        Авторасстановка
      </button>
      <button
        className={styles.button}
        onClick={resetField}
      >
        Сбросить
      </button>
    </div>
  </div>

  <div className={styles['battlefield-wrapper']}>
    <Battlefield
      matrix={matrixState}
      onCellClick={handleCellClick}
      selectedShip={selectedShip}
    />

    <div className={styles['under-battlefield-buttons-
container']}>
      <Button
        color='#2ecc71'
        hoverColor='#27ae60'
        disabled={Object.values(availableShips).some(count => count > 0)}
        className={styles['submit-button']}
        onClick={handleStartGame}
      >
        Начать игру
      </Button>

      <Button
        color="var(--color-green)"
        hoverColor="var(--color-green-dark)"
        onClick={() => navigate('/') }
        className={styles['to-menu-button']}
      >
        В меню
      </Button>
    </div>
  </div>
</div>
);
};

export default SetupModule;

```

Модуль для игры с ботом

```
import {SeaFightEngine} from "@/engines/seaFightEngine.ts";
import {useCallback, useEffect, useRef, useState} from "react";
import {Bot} from "@/engines/botEngine.ts";
import type {BattlefieldMatrix} from "@/types/ship.types.ts";
import {GameModule} from "@/modules/game-module";
import styles from "../styles/BotGameModule.module.css";
import {AudioPlayer} from "@/components/audio-player";
import type {GameStatus} from "@/types/game.types.ts";

interface BotGameModuleProps {
  filledPlayerField: BattlefieldMatrix;
}

const BotGameModule = ({filledPlayerField} : BotGameModuleProps) => {
  const [playerField, setPlayerField] =
    useState<BattlefieldMatrix>(filledPlayerField);
  const [opponentField, setOpponentField] =
    useState<BattlefieldMatrix>(SeaFightEngine.generateField());
  const [isPlayerTurn, setIsPlayerTurn] = useState(true);
  const [gameStatus, setGameStatus] =
    useState<GameStatus>('playing');
  const [soundEnabled, setSoundEnabled] = useState(true);

  const botFieldEngine = useRef(new
    SeaFightEngine(opponentField)).current;
  const bot = useRef(new Bot(playerField, setPlayerField)).current;

  const toggleSound = useCallback(() => {
    setSoundEnabled(prev => !prev);
  }, []);

  const { playSound } = AudioPlayer();
  useEffect(() => {
    if (!soundEnabled) return;

    switch (gameStatus) {
      case 'won': playSound('win'); break;
      case 'lost': playSound('lose'); break;
      case 'opponentLeave': playSound('opponentLeave'); break;
    }
  }, [gameStatus, playSound, soundEnabled]);

  const handlePlayerShot = useCallback((row: number, col: number) =>
  {
    if (!isPlayerTurn || gameStatus !== 'playing') return;

    const wasHit = botFieldEngine.shoot(row, col);
    if (soundEnabled) playSound('boom');
    setOpponentField([...botFieldEngine.getField()]);

    if (botFieldEngine.areAllShipsDestroyed()) {
```



```

        setStatus('won');
        return;
    }

    if (wasHit) return;
    setIsPlayerTurn(false);
    handleBotTurn();
}, [isPlayerTurn, gameStatus, botFieldEngine, bot, soundEnabled,
playSound]);

const handleBotTurn = useCallback(() => {
    const timer = setTimeout(() => {
        const botHit = bot.makeShoot();
        if (soundEnabled) playSound('boom');

        if (bot.isWin()) {
            setStatus('lost');
            return;
        }

        if (botHit) {
            handleBotTurn();
        } else {
            setIsPlayerTurn(true);
        }
    }, 500);

    return () => clearTimeout(timer);
}, [bot, soundEnabled, playSound]);

return (
    <div className={styles['online-game-module-wrapper']}>
        <GameModule
            playerField={playerField}
            onPlayerShot={handlePlayerShot}
            opponentField={opponentField}
            isPlayerTurn={isPlayerTurn}
            gameStatus={gameStatus}
            soundEnabled={soundEnabled}
            onSoundToggle={toggleSound}
        />
    </div>
);
};

export default BotGameModule;

```

Модуль для игры онлайн

```

import styles from './styles/OnlineGameModule.module.css'
import {useCallback, useEffect, useRef, useState} from 'react';
import { GameModule } from "@modules/game-module";
import type { BattlefieldMatrix } from "@types/ship.types.ts";
import { SeaFightEngine } from "@engines/seaFightEngine.ts";
import { useSeaBattleHub } from '@hooks/useSeaBattleHub';

```

```

import type {GameStatus} from "@types/game.types.ts";
import {Chat} from "@components/chat";
import {AudioPlayer} from "@components/audio-player";

interface OnlineGameModuleProps {
  filledPlayerField: BattlefieldMatrix;
  userName: string;
}

const OnlineGameModule = ({ filledPlayerField, userName}:
OnlineGameModuleProps) => {
  const [playerField, setPlayerField] =
useState<BattlefieldMatrix>(filledPlayerField);
  const [opponentField, setOpponentField] =
useState<BattlefieldMatrix>(SeaFightEngine.createEmptyField);
  const [isPlayerTurn, setIsPlayerTurn] = useState(false);
  const [gameStatus, setGameStatus] =
useState<GameStatus>('waiting');

  const opponentFieldEngine = useRef<SeaFightEngine>(new
SeaFightEngine(opponentField));
  const playerFieldEngine = useRef(new
SeaFightEngine(playerField)).current;

  const {
    session,
    connection,
    chatMessages,
    sendBattlefield,
    makeMove,
    sendChatMessage,
    sendPlayerName
  } = useSeaBattleHub(
    useCallback((isPlayerTurn: boolean) => {
      setIsPlayerTurn(isPlayerTurn);
      setGameStatus('playing');
    }, []),
    setOpponentField,
    setGameStatus,
  );

  // Отправляем имя игрока при создании сессии
  useEffect(() => {
    if (session && gameStatus === 'waiting') {

      sendPlayerName(session.sessionId, userName || "Игрок");
      sendBattlefield(session.sessionId, playerField);
    }
  }, [session, gameStatus, playerField, sendBattlefield,
sendPlayerName]);

  // Отправляем поле на сервер при создании сессии
  useEffect(() => {
    if (session && gameStatus === 'waiting') {

```

```

        sendBattlefield(session.sessionId, playerField);
    }
}, [session, gameStatus, playerField, sendBattlefield]);

// Эффект для обновления движка при изменении opponentField
useEffect(() => {
    opponentFieldEngine.current = new
SeaFightEngine(opponentField);
}, [opponentField]);

// Настройка звуков
const [soundEnabled, setSoundEnabled] = useState(true);

const toggleSound = useCallback(() => {
    setSoundEnabled(prev => !prev);
}, []);
const { playSound } = AudioPlayer();
const [prevGameStatus, setPrevGameStatus] =
useState<GameStatus>('waiting');

useEffect(() => {
    if (!soundEnabled || gameStatus === prevGameStatus) return;

    switch (gameStatus) {
        case 'won': playSound('win'); break;
        case 'lost': playSound('lose'); break;
        case 'opponentLeave': playSound('opponentLeave'); break;
    }

    setPrevGameStatus(gameStatus);
}, [gameStatus, playSound, soundEnabled, prevGameStatus]);

// Обработка хода противника
const handleOpponentTurn = useCallback((row: number, col: number)
=> {
    if (gameStatus !== 'playing') return;

    // Обрабатываем выстрел противника
    const wasHit = playerFieldEngine.shoot(row, col);
    if (soundEnabled) playSound('boom');
    setPlayerField([...playerFieldEngine.getField()]);

    // Проверяем поражение
    if (playerFieldEngine.areAllShipsDestroyed()) {
        setGameStatus('lost');
        return;
    }

    // Если противник промахнулся, наш ход
    if (!wasHit) {
        setIsPlayerTurn(true);
    }
}, [gameStatus, playerFieldEngine, soundEnabled]);

```

```

// Настройка обработчиков сообщений от сервера
useEffect(() => {
  if (!connection) return;

  connection.on('ReceiveOpponentMove', (move: {row: number,
column: number}) => {
    handleOpponentTurn(move.row, move.column);
  });

  return () => {
    connection.off('ReceiveOpponentMove');
  };
}, [connection, handleOpponentTurn]);

const handlePlayerShot = useCallback((row: number, col: number) =>
{
  if (!isPlayerTurn || gameStatus !== 'playing' || !session)
return;

  // Игрок делает выстрел
  const wasHit = opponentFieldEngine.current.shoot(row, col);
  if (soundEnabled) playSound('boom');
  setOpponentField([...opponentFieldEngine.current.getField()]);

  // Отправляем ход на сервер
  makeMove(session.sessionId, row, col);

  // Проверяем победу
  if (opponentFieldEngine.current.areAllShipsDestroyed()) {
    setGameStatus('won');
    return;
  }

  if (wasHit) return; // Если игрок попал, он ходит еще раз
  setIsPlayerTurn(false); // Иначе ход противника
}, [isPlayerTurn, gameStatus, session, makeMove,
opponentFieldEngine, soundEnabled]);

return (
  <div className={styles['online-game-module-wrapper']}>
    <GameModule
      playerField={playerField}
      onPlayerShot={handlePlayerShot}
      opponentField={opponentField}
      isPlayerTurn={isPlayerTurn}
      gameStatus={gameStatus}
      soundEnabled={soundEnabled}
      onSoundToggle={toggleSound}
    />

    {(((gameStatus !== 'waiting') && (gameStatus !==
'opponentLeave')) &&
      <Chat

```

```

                                onSendMessage={(message) => session &&
sendChatMessage(session.sessionId, message)}
                                messages={chatMessages}
                                />
                                )}
                                </div>
                                );
};

export default OnlineGameModule;

```

Реализация движка для игры в морской бой

```

import type {BattlefieldMatrix, ShipDirection, ShipSize} from
"@/types/ship.types.ts";

const FIELD_SIZE = 10;

export class SeaFightEngine {
  private field: BattlefieldMatrix;

  constructor(initialField?: BattlefieldMatrix) {
    this.field = initialField || SeaFightEngine.createEmptyField();
  }

  public getField(): BattlefieldMatrix {
    return this.field.map(row => [...row]);
  }

  public shoot(row: number, col: number): boolean {
    if (row < 0 || row >= FIELD_SIZE || col < 0 || col >= FIELD_SIZE) {
      return false;
    }

    if (this.field[row][col] === 'ship') {
      this.field[row][col] = 'hit';

      // Проверяем, уничтожен ли корабль полностью
      if (this.isShipDestroyed(row, col)) {
        this.markDestroyedShipPerimeter(row, col);
      } else {
        // Помечаем углы как FORBIDDEN
        this.markCornersForbidden(row, col);
      }
      return true;
    } else {
      // Пустая клетка
      this.field[row][col] = 'miss';
      return false;
    }
  }

  public areAllShipsDestroyed(): boolean {
    for (let i = 0; i < FIELD_SIZE; i++) {
      for (let j = 0; j < FIELD_SIZE; j++) {

```

```

        if (this.field[i][j] === 'ship') {
            return false;
        }
    }
    return true;
}

public getEmptyAroundCells(row: number, col: number): number[] {
    const emptyCells: number[] = [];
    // Проверяем клетку слева
    if (col > 0 && this.isCellShootable(row, col-1)) {
        emptyCells.push(row * 10 + col - 1);
    }
    // Проверяем клетку справа
    if (col < FIELD_SIZE - 1 && this.isCellShootable(row, col+1)) {
        emptyCells.push(row * 10 + col + 1);
    }
    // Проверяем клетку сверху
    if (row > 0 && this.isCellShootable(row - 1, col)) {
        emptyCells.push((row - 1) * 10 + col);
    }
    // Проверяем клетку снизу
    if (row < FIELD_SIZE - 1 && this.isCellShootable(row+1, col)) {
        emptyCells.push((row + 1) * 10 + col);
    }

    return emptyCells;
}

public isCellShootable (row: number, col: number): boolean {
    return this.field[row][col] === 'empty' || this.field[row][col] ===
'ship';
}

private markCornersForbidden(row: number, col: number): void {
    // Левый верхний угол
    if (row > 0 && col > 0 && this.field[row - 1][col - 1] === 'empty') {
        this.field[row - 1][col - 1] = 'forbidden';
    }

    // Правый верхний угол
    if (row > 0 && col < FIELD_SIZE - 1 && this.field[row - 1][col + 1]
=== 'empty') {
        this.field[row - 1][col + 1] = 'forbidden';
    }

    // Левый нижний угол
    if (row < FIELD_SIZE - 1 && col > 0 && this.field[row + 1][col - 1]
=== 'empty') {
        this.field[row + 1][col - 1] = 'forbidden';
    }

    // Правый нижний угол

```

```

        if (row < FIELD_SIZE - 1 && col < FIELD_SIZE - 1 && this.field[row +
1][col + 1] === 'empty') {
            this.field[row + 1][col + 1] = 'forbidden';
        }
    }

    private isShipDestroyed(row: number, col: number): boolean {
        // Проверяем все клетки корабля (может быть горизонтальным или
вертикальным)
        // Проверяем влево
        let r = row;
        let c = col;
        while (c >= 0 && (this.field[r][c] === 'ship' || this.field[r][c] ===
'hit')) {
            if (this.field[r][c] === 'ship') return false;
            c--;
        }

        // Проверяем вправо
        c = col + 1;
        while (c < FIELD_SIZE && (this.field[r][c] === 'ship' ||
this.field[r][c] === 'hit')) {
            if (this.field[r][c] === 'ship') return false;
            c++;
        }

        // Проверяем вверх
        r = row - 1;
        c = col;
        while (r >= 0 && (this.field[r][c] === 'ship' || this.field[r][c] ===
'hit')) {
            if (this.field[r][c] === 'ship') return false;
            r--;
        }

        // Проверяем вниз
        r = row + 1;
        while (r < FIELD_SIZE && (this.field[r][c] === 'ship' ||
this.field[r][c] === 'hit')) {
            if (this.field[r][c] === 'ship') return false;
            r++;
        }

        return true;
    }

    private markDestroyedShipPerimeter(row: number, col: number): void {
        // Находим все клетки корабля
        const shipCells: [number, number][] = [];
        this.findAllShipCells(row, col, shipCells);

        // Помечаем периметр
        for (const [r, c] of shipCells) {
            for (let i = Math.max(0, r - 1); i <= Math.min(FIELD_SIZE - 1, r
+ 1); i++) {

```

```

        for (let j = Math.max(0, c - 1); j <= Math.min(FIELD_SIZE -
1, c + 1); j++) {
            if (this.field[i][j] === 'empty') {
                this.field[i][j] = 'forbidden';
            }
        }
    }
}

private findAllShipCells(
    row: number,
    col: number,
    foundCells: [number, number][]
): void {
    if (
        row < 0 ||
        row >= FIELD_SIZE ||
        col < 0 ||
        col >= FIELD_SIZE ||
        this.field[row][col] !== 'hit' ||
        foundCells.some(([r, c]) => r === row && c === col)
    ) {
        return;
    }

    foundCells.push([row, col]);

    // Проверяем соседние клетки
    this.findAllShipCells(row - 1, col, foundCells); // вверх
    this.findAllShipCells(row + 1, col, foundCells); // вниз
    this.findAllShipCells(row, col - 1, foundCells); // влево
    this.findAllShipCells(row, col + 1, foundCells); // вправо
}

```

```

    public static canPlaceShip = (row: number, col: number, size: ShipSize,
direction: ShipDirection, matrix: BattlefieldMatrix): boolean => {
    // Проверяем границы поля
    if (direction === 'horizontal') {
        if (col + size > 10) return false;
    } else {
        if (row + size > 10) return false;
    }

    // Проверяем что все клетки свободны
    for (let i = 0; i < size; i++) {
        const cellRow = direction === 'horizontal' ? row : row + i;
        const cellCol = direction === 'horizontal' ? col + i : col;

        if (matrix[cellRow][cellCol] !== 'empty') return false;
    }
}

```



```

        // Проверяем соседние клетки
        for (let r = Math.max(0, cellRow - 1); r <= Math.min(9, cellRow +
1); r++) {
            for (let c = Math.max(0, cellCol - 1); c <= Math.min(9,
cellCol + 1); c++) {
                if (matrix[r][c] === 'ship') return false;
            }
        }
    }

    return true;
};

public static generateField = () => {
    const newMatrix: BattlefieldMatrix = Array(10).fill(null).map(() =>
Array(10).fill('empty'));
    const shipsToPlace: ShipSize[] = [4, 3, 3, 2, 2, 2, 1, 1, 1, 1];

    // Пытаемся расставить каждый корабль
    for (const shipSize of shipsToPlace) {
        let placed = false;
        let attempts = 0;

        while (!placed && attempts < 100) {
            attempts++;
            const direction: ShipDirection = Math.random() > 0.5 ?
'horizontal' : 'vertical';
            const row = Math.floor(Math.random() * 10);
            const col = Math.floor(Math.random() * 10);

            if (SeaFightEngine.canPlaceShip(row, col, shipSize,
direction, newMatrix)) {
                // Ставим корабль
                for (let i = 0; i < shipSize; i++) {
                    if (direction === 'horizontal') {
                        newMatrix[row][col + i] = 'ship';
                    } else {
                        newMatrix[row + i][col] = 'ship';
                    }
                }
                placed = true;
            }
        }
    }

    return newMatrix;
}

public static createEmptyField(): BattlefieldMatrix {
    return Array(FIELD_SIZE)
        .fill(null)
        .map(() => Array(FIELD_SIZE).fill('empty'));
}
}

```